Tarn

Protocol Specification

Wire format, key hierarchy, storage layout

May 2026

Contents

Tarn Protocol — Working Draft

Status: Active design discussion (Issue #69) **Last updated:** 2026-05-04 (single-envelope cleanup — legacy KDF/envelope versions cut, current envelope renumbered to v1)

Tarn is an app-agnostic platform for storing encrypted, user-owned data permanently on Arweave. This document defines the complete protocol: identity, authentication, encryption, and data operations.

App view vs. protocol view

This document describes the **wire protocol** — what bytes go on Arweave, what tags identify what, what HTTP shapes the API speaks, what the encryption envelopes look like, how share-log seq mechanics work. Most app developers never need to read it.

The **app-facing SDK** lives in `client/`. It exposes typed CRUD per collection, friend connections, sharing, recovery, and account management — without app code touching tags, txids, encryption envelopes, share-log sequence numbers, or HPKE handshakes.

Read this document when you are:

- **building a new client or server in a different language.** Everything you need to interoperate with Tarn at the wire level is here.
- **debugging a low-level interaction.** When the SDK abstraction frays (it shouldn't, but it sometimes does), the protocol spec is the source of truth.
- **building the "always access your data" recovery client.** The end-game promise of Tarn is that any client with the user's credentials can decrypt their data straight from Arweave with no Tarn API in the loop. That client reads schemas, content blobs, and share logs directly from Arweave gateways via GraphQL. It implements this document.

For ordinary app development, start with the SDK README. Come back here when you actually need wire-level detail.

Key Hierarchy

All sub-keys are derived using **HKDF-Expand** (RFC 5869) with structured info strings. No ad-hoc SHA-256 concatenation. The info string follows a fixed structure:

```
info = protocol || purpose || app_id || version || 0x01
```

Where:

- `protocol` = `"tarn"` (fixed)
- `purpose` = one of: `"lookup"`, `"encrypt"`, `"sign"`

- `app_id` = registered app identifier (e.g., `"bookish"`)
- `version` = `"1"` (derivation version — allows future KDF changes)
- `0x01` = HKDF counter byte per RFC 5869

HKDF-Expand with a single 32-byte output block reduces to one HMAC call:

```
sub_key = HMAC-SHA256(master_key, info)
```

Full derivation chain

```
username + password (user input, never leaves client)
  -> master_key                  Argon2id(password, SHA-256(normalizedUsername),
    m=64MiB, t=3, p=1)

master_key + app_id:
  -> credential_lookup_key       HMAC-SHA256(master_key, "tarn" || "lookup"  || app_id
    || "1" || 0x01)
  -> credential_encryption_key   HMAC-SHA256(master_key, "tarn" || "encrypt" || app_id
    || "1" || 0x01)
  -> signing_key_seed            HMAC-SHA256(master_key, "tarn" || "sign"    || app_id
    || "1" || 0x01)
     -> signing_key_pair         ECDSA P-256 from signing_key_seed (see P-256
    derivation)
       -> public_key             sent to API, stored in D1 and on Arweave
       -> private_key            never leaves client
```

Master-key KDF

Argon2id is the only supported KDF. Parameters: m=64 MiB, t=3, p=1, hash length 32 bytes. Salt is `SHA-256(normalizedUsername)` — deterministic from the account identifier, no per-account salt at this layer (the recovery factor's `Argon2id(phrase, recovery_salt)` does carry a random salt; see Recovery factor).

Parameters were chosen against a ~2s slow-device login budget: 64 MiB memory + 3 iterations + 1-way parallelism (single-threaded; aligns with browser realities). The memory-hard property neutralizes GPU/ASIC parallelism in line with the OWASP Argon2id recommendation.

> ***Note on prior KDFs.*** *Earlier drafts of Tarn supported PBKDF2-SHA256 (600K iters) as a legacy path, with login dispatch trying Argon2id first and falling back. That back-compat was cut cleanly when Tarn still had a single user — the only KDF clients ever derive is Argon2id. There is no dispatch.*

Username field — format-agnostic

Tarn does not validate the username's format. It's a UTF-8 string used as a KDF salt input (after `trim().toLowerCase()` normalization) and as a public user-lookup key for connection bootstrap. Apps choose whether to enforce email-shape, handle-shape, phone-shape, or anything else. The implementation neutrally calls the parameter `username`; many apps will populate it with an email address, but Tarn does not require this. Earlier protocol drafts called this field `email` and that name still appears in some legacy on-Arweave records (see Connection record back-compat below); the wire-protocol field name is now `username`.

Per-app isolation: Every derived key includes `app_id`. The same username+password produces completely independent identities per app. Different credential_lookup_key, different encryption_key, different signing key. A Bookish user and a Cellar user with the same username+password cannot see each other's data, share sessions, or even detect each other's existence.

P-256 private key derivation

The `signing_key_seed` (32 bytes from HMAC) is used as the P-256 private scalar. P-256 requires the private key to be in [1, n-1] where n is the curve order.

```
seed = HMAC-SHA256(master_key, "tarn" || "sign" || app_id || "1" || 0x01)
if seed == 0 or seed >= n:
  seed = HMAC-SHA256(master_key, "tarn" || "sign" || app_id || "1" || 0x02)
  (repeat with incrementing counter — probability of needing this: ~2^-128)
```

Import as PKCS#8 DER → export public key as SPKI → send SPKI to API.

Wrapped Data Key

The `data_encryption_key` (DEK) is wrapped using **AES-KW** (RFC 3394, Key Wrap):

```
wrapped_bytes = AES-KW-Wrap(data_encryption_key, credential_encryption_key)
                            ^^^^ payload            ^^^^ wrapping key
```

AES-KW is deterministic (no IV), purpose-built for key wrapping, and available in WebCrypto via `crypto.subtle.wrapKey('raw', key, wrappingKey, 'AES-KW')`.

At registration the DEK is generated as 32 fresh random bytes — no self-wrapping. The DEK is wrapped twice (once under the password factor's KEK, once under the recovery factor's KEK) and packaged into a v1 envelope (see below).

After a credential change, every existing chain entry is re-wrapped under the new `credential_encryption_key`, and a fresh DEK is appended at the next-highest generation — see Forward-secret DEK rotation below.

Wire format (`wrapped_data_key` field)

The API stores `wrapped_data_key` as opaque text. The string is a self-describing JSON envelope; the client validates `v` and `kdf` on read and rejects anything else.

v1 — multi-factor DEK chain. Each chain entry is wrapped under one or more factors; any factor's KEK independently unwraps the DEK. Three factor types exist: `"password"` and `"recovery_phrase"` (mandatory — every account carries both) and `"passkey_prf"` (Phase 6, opt-in — zero or more per account, one per registered passkey). The `recovery` block holds the recovery KDF's params + per-account salt and is required.

```
{
  "v": 1,
  "kdf": "argon2id",
  "kdf_params": { "m_kib": 65536, "t": 3, "p": 1 },
  "recovery": {
    "kdf": "argon2id",
```

```
    "kdf_params": { "m_kib": 65536, "t": 3, "p": 1 },
    "salt": "<base64 16-byte per-account random salt>"
  },
  "dek_chain": [
    { "gen": 1, "wrappings": [
      { "factor": "password",        "wrapped": "<base64 AES-KW ciphertext (40 bytes)>"
    },
      { "factor": "recovery_phrase", "wrapped": "<base64 AES-KW ciphertext (40 bytes)>"
    },
      { "factor": "passkey_prf",     "credential_id": "<base64url credential ID>",
                                    "wrapped": "<base64 AES-KW ciphertext (40 bytes)>"
    }
    ]},
    { "gen": 2, "wrappings": [
      { "factor": "password",        "wrapped": "<base64 AES-KW ciphertext (40 bytes)>"
    }
    ]}
  ]
}
```

• Each `dek_chain` entry's `wrappings` array carries the same DEK wrapped under each factor's KEK. AES-KW is deterministic, so the same (DEK, KEK) pair always produces the same ciphertext bytes — preserving register-retry idempotency.

• `passkey_prf` wrappings carry an additional `credential_id` field (base64url-encoded WebAuthn credential ID) so the SDK can pick the right wrapping when more than one passkey is registered. Within a single `wrappings` array, `(factor, credential_id)` must be unique — duplicate `passkey_prf` entries with the same credential ID are rejected at parse time. `password` and `recovery_phrase` carry no credential ID and may appear at most once per gen.

• When `changeCredentials` runs without the account key (caller passed `acceptRecoveryGap: true`): old gens preserve their existing recovery wrappings verbatim (re-wrapping under the same KEK is byte-identical anyway), and the new gen N+1 has only a `password` wrapping. The gap is closed by `recoverAccount` or by `rotateAccountKey` (Phase 4 — `POST /api/v1/account/rotate-account-key`, exposed as `tarn.accountKey.rotate()`), both of which re-wrap every gen under the recovery factor. By default the SDK requires `phrase` and refuses the rotation if it would create a gap.

• Passkey wrappings are preserved verbatim by every envelope-mutating SDK path (`changeCredentials`, `rotateAccountKey`, `recoverAccount`) since AES-KW is deterministic and the underlying DEKs do not change. The new gen N+1 created by `changeCredentials` does NOT receive a passkey wrapping (the SDK does not have the PRF outputs in that flow); the user must re-register each passkey for new-gen data to be passkey-unwrappable. `rotateAccountKey` and `recoverAccount` do NOT mint a new gen, so they preserve passkey unwrappability across the rotation.

• The current generation (used for new writes) is the entry with the highest `gen`. Old gens stay in the chain so older content blobs remain decryptable.

The envelope is byte-stable for the same `(username, password, app, recovery_phrase, recovery_salt)` inputs (AES-KW is deterministic; `JSON.stringify` is insertion-ordered; chain entries are written in generation order). This preserves the register-retry idempotency check (server compares the stored `wrapped_data_key` to the incoming one byte-for-byte; a retry of an interrupted register sends the same bytes).

> ***Note on prior envelope versions.*** *Earlier drafts supported a v1 bare-base64 single-key shape (PBKDF2-era), a v2 single-key JSON envelope (early Argon2id), and a v3 single-factor chain envelope (forward-secret rotation pre-recovery-factor). The `v` field was renumbered to `1` after those legacy paths were cut, so the current shape's `v: 1` is the post-cleanup definition above — not the pre-cleanup bare-base64 shape.*

Forward-secret DEK rotation

On every credential change, the client mints a fresh random DEK at gen N+1 and appends it to the chain. Subsequent writes use the new gen; old gens remain in the chain so prior data is still readable. An attacker who later compromises the OLD `credential_encryption_key` cannot decrypt content written after the rotation (the new gen DEK is not derivable from old credentials).

This is the only forward-secrecy property Tarn provides today. Old credential blobs on Arweave remain unwrappable by anyone who held the old credentials, but the DEK they yield is bound to data written before the rotation.

Recovery factor

Every account publishes a `recovery_lookup_key` and `recovery_public_key` alongside the password-derived `credential_lookup_key` / `public_key`. Both are derived from the user's BIP39 account key (24-word, 256-bit entropy) and let the user authenticate to the API for credential rotation when they have lost their password.

Derivation:

```
phrase_entropy           = BIP39 entropy bytes (32 for 24-word account key)
recovery_lookup_key      = HMAC-SHA256(phrase_entropy, "tarn" || "recovery-lookup" ||
    app_id || "1" || 0x01)
recovery_signing_seed    = HMAC-SHA256(phrase_entropy, "tarn" || "recovery-sign"   ||
    app_id || "1" || 0x01)
recovery_signing_key     = ECDSA P-256 from recovery_signing_seed (same retry rule as
    the password-derived signing seed)
recovery_KEK             = Argon2id(account_key, recovery_salt, m=64MiB, t=3, p=1)
                           (recovery_salt is per-account random, lives in the
    envelope's `recovery.salt`)
```

`recovery_lookup_key` and `recovery_signing_key` derive from the raw account-key entropy directly (no salt), so they are stable across credential changes — the account key remains the same secret regardless of how many times the password rotates. The `recovery_KEK` derives via Argon2id with the per-account salt, providing the slow-brute-force defense at unwrap time.

> ***Note on naming.*** *The wire-protocol identifiers all use the historical "recovery" / "recovery_phrase" terminology (`recovery_lookup_key`, `recovery_public_key`, the `"recovery_phrase"` factor string, `FACTOR_RECOVERY_PHRASE`, and HMAC info-string*

components like `"recovery-lookup"`). The user-facing term in the SDK and product surfaces is "account key" — the secret IS the account credential, not an optional fallback. The wire identifiers are deliberately frozen to preserve compatibility; only human-facing surfaces use the new term.

The account key is **mandatory at signup** — the SDK enforces this with a synchronous `recoveryAcknowledged: true` flag on `register()`. The kit (PDF or structured JSON) is rendered entirely on the client; Tarn never sees the account key, the entropy, the KEK, or the rendered PDF. Apps are responsible for surfacing the kit to the user (download, print, or any out-of-band channel the app implements). Tarn does not provide email delivery or any other transport for account-key material — that would require Tarn to handle plaintext kit bytes, which is incompatible with the zero-knowledge framing.

Account-key storage models — Model A vs Model B

Two distinct storage modes exist for the account key. The wire protocol supports both; an app picks one per registration by sending or omitting a `wrapped_account_key` field in the register payload. There is no separate protocol flag — the field's presence is the signal.

Model A — no backup stored. Tarn never stores any form of the account key. The user holds the only copy (printed, in a password manager, etc.). The `wrapped_account_key` field is absent at registration and remains null in the `accounts` row.

- Lose your password AND your account key → data is permanently inaccessible. Even Tarn cannot help; nothing on the server contains the key material.
- **Credential compromise alone does NOT yield the account key.** A phished password authenticates to Tarn and decrypts content (via the password-side DEK chain), but the account key itself is not derivable from anything Tarn holds. To obtain the account key, the attacker must compromise some separate channel where the user actually stored it (their password manager, their physical safe, the email they sent to themselves, etc.).
- This is the strict zero-knowledge posture and the historical Tarn default.

Model B — encrypted backup stored. At registration, the client wraps the account key under the gen-1 DEK (`wrapped_account_key = AES-GCM(key=DEK_gen1, plaintext=account_key_utf8, aad="tarn-wrapped-account-key-v1")`) and sends the ciphertext as `wrapped_account_key` in the register payload. The API stores it on the `accounts` row and publishes it as part of the account record on Arweave (so the "Tarn infra is rebuildable from Arweave" property holds).

- A logged-in user can retrieve and view the account key from app settings at any time. The retrieval flow is gated by step-up auth (re-derived password proof) and decryption happens client-side: fetch the ciphertext, derive DEK from the (re-entered) password, AES-GCM-decrypt locally.
- **Credential compromise DOES yield the account key.** A phished password derives the DEK, the DEK decrypts the wrap, the attacker has the account key. The account key in this model is no longer a password-independent lifeline — it is a credential-recoverable item like everything else, modulo whatever authorization gate sits in front of fetch (step-up at minimum;

apps may layer additional out-of-band challenges).

• Trade-off: this dramatically narrows the population that loses access through "saved my password but not the key, now I can't find the key." The cost is the model shift above.

Apps choose by sending or not sending `wrapped_account_key` **at registration.** The wire format is identical otherwise.

Users can switch later. Toggle endpoints `PUT /api/v1/account/account-key` (enable, Model A → B) and `DELETE /api/v1/account/account-key` (disable, Model B → A) are documented below — both require the same JWT + step-up token posture as the fetch endpoint. Either toggle republishes the credential blob to Arweave so the rebuild path stays in sync. The SDK exposes them as `tarn.accountKey.enableKeyStorage()` and `tarn.accountKey.disableKeyStorage()`.

Passkey support is independent of this choice. Passkeys (Phase 6) add WebAuthn-PRF as a third auth factor in the envelope (alongside `password` and `recovery_phrase`). Whether or not an account uses Model A or Model B, it can independently opt in to passkey factors. See Passkey factor (Phase 6) below.

Wire format details (Phase 3)

Wrap construction. The client computes:

```
wrapped_account_key = AES-GCM(
  key       = DEK_gen1,
  plaintext = utf8(account_key),
  AAD       = utf8("tarn-wrapped-account-key-v1"),
)
```

Wire bytes are `IV(12) || ciphertext+GCM-tag(N+16)`, then base64-encoded. The AAD string `"tarn-wrapped-account-key-v1"` is part of the wire format — any future revision of this wrap MUST bump the version suffix and reject the old AAD on read. Decrypts with a different (or absent) AAD MUST fail.

The server validates only that the field is a base64 string of plausible length (100..1024 chars) and stores it verbatim. Server cannot decrypt — DEK_gen1 is derived from the user's password.

`account_key_stored` **indicator.** The `/auth/verify` response carries an `account_key_stored: boolean` field for user-role logins (omitted for app-role JWTs). It reflects whether the row's `wrapped_account_key` is non-null. Apps render the appropriate Settings UI based on this (`true` → "view your account key"; `false` → "no backup stored — enable in settings"). The wrap itself is NOT included on `/auth/verify` — fetching it is a separately gated operation, see below.

Step-up auth: `POST /api/v1/auth/step-up`

Issues a short-lived single-use token authorizing one privileged operation. Currently scoped to `account_key_fetch` (the wrap retrieval below); future privileged endpoints (Phase 4 toggles, etc.) reuse the same machinery with new scope strings.

Request body (same shape as `/auth/verify`'s password-side path):

```
{
  "credential_lookup_key": "<64-char hex>",
  "nonce": "<64-char hex from /auth/challenge>",
  "signature": "<base64 ECDSA P-256 signature over the nonce>",
  "scope": "account_key_fetch"
}
```

The flow: client calls `/auth/challenge` with `credential_lookup_key`, derives `credential_signing_key` from the freshly re-entered password, signs the nonce, posts to `/auth/step-up` with the same `credential_lookup_key`, the consumed nonce, the signature, and the scope. The signature verifies against the row's stored `public_key` exactly like `/auth/verify`. Recovery-flow auth (`recovery_lookup_key`) is intentionally NOT supported — re-entering the account key yields a recovery JWT via the existing `/auth/verify`, not a step-up token.

Response:

```
{
  "step_up_token": "<64-char hex random token>",
  "expires_at": <unix-millis>,
  "scope": "account_key_fetch"
}
```

Token mechanism. Opaque random 32-byte hex strings stored in the `step_up_tokens` D1 table with a 60-second TTL and a `consumed_at` single-use guard. The single-use guard is enforced atomically via `UPDATE ... WHERE consumed_at IS NULL ... RETURNING ...` — the first reader's UPDATE matches and returns; concurrent re-uses match zero rows and read null. Chosen over JWT-with-revocation-list because the table stays trivially small (60s TTL) and there's no extra signing key to thread through.

Account-key fetch: `GET /api/v1/account/account-key`

Returns the stored wrap. Requires BOTH:

- `Authorization: Bearer <session JWT>` — proves the caller is logged into *some* account.
- `X-Step-Up-Token: <token>` — proves a fresh password re-entry on this device.

The server cross-checks that both bind to the same `data_lookup_key` (defense in depth against a JWT/token mis-binding). Either auth missing → 401.

Response on success (Model B account):

```
{
  "wrapped_account_key": "<base64 ciphertext>",
  "recovery_salt": "<base64 16 bytes — Argon2id salt from the v1 envelope>",
  "kdf_params": { "m_kib": 65536, "t": 3, "p": 1 },
  "recovery_lookup_key": "<64-char hex — pinning check value>"
}
```

Response on Model A account (no wrap stored): `404` with body `{"error": "no_account_key_stored"}` so the SDK can render the appropriate UI without conflating with "wrong account."

Wrap-pinning check (client-side). After decrypting the wrap, the SDK derives `recovery_lookup_key` from the resulting account-key entropy via the same HMAC chain used elsewhere (`HMAC-SHA256(entropy, "tarn-v1:recovery-lookup:<app_id>:1")` truncated to 32 bytes). It compares this to the `recovery_lookup_key` returned in the fetch response. Mismatch → SDK throws a typed `AccountKeyPinningError` and refuses to return the phrase. The pin check defends against the server returning a wrap that decrypts to a *different* (also valid) 24-word phrase — possible under a colluding storage backend or a wrap mis-binding bug.

Audit log. Every successful fetch writes a row to `account_key_fetch_log` (`data_lookup_key` , `fetched_at` , hashed `ip_hash` , truncated `user_agent` , `op = 'fetch'`). Inserted via `ctx.waitUntil` so a D1 hiccup does not block the response. This enables future "account key was last viewed at X" UX in app settings. Phase 4 reuses the same table to record `op = 'enable' | 'disable' | 'rotate'` rows for the toggle and rotation endpoints below — same columns, same semantics, just a discriminator on `op` . The historical name `account_key_fetch_log` is preserved for tool compatibility.

Account-key storage toggle (Phase 4)

Enable storage (Model A → Model B): `PUT /api/v1/account/account-key`

Stores a fresh wrap on the `accounts` row and republishes the credential blob to Arweave. Auth: BOTH a session JWT AND a fresh step-up token (same posture as the fetch — the user just re-entered their password to derive the gen-1 DEK that produced the wrap; the step-up proves they did).

Request body:

```
{ "wrapped_account_key": "<base64 ciphertext, 100..1024 chars>" }
```

Response: `200 OK` with `{ "stored": true }` . The same validation rules as the registration `wrapped_account_key` field apply (length bounds, base64 / base64url charset). If the column is already populated this is treated as an OVERWRITE — the SDK's `enableKeyStorage` does a client-side wrap-pinning check before sending, so calling this twice with valid input is byte-equivalent to calling it once. Audit row: `op = 'enable'` .

Errors: `400` for missing/invalid wrap, `401` for missing/invalid JWT or step-up token, `403` for app-role JWTs.

Disable storage (Model B → Model A): `DELETE /api/v1/account/account-key`

Sets `accounts.wrapped_account_key = NULL` and republishes the credential blob to Arweave. Same auth posture (JWT + step-up).

Response on a Model B account: `200 OK` with `{ "stored": false }` . Response on an already-Model-A account: `200 OK` with `{ "stored": false, "already_disabled": true }` — idempotent by design. UI code can treat both responses identically; the `already_disabled` flag is purely informational. Audit row: `op = 'disable'` (only on the actual write path, not the no-op).

Errors: `401` for missing/invalid JWT or step-up token, `403` for app-role JWTs.

Account-key rotation (Phase 4): `POST /api/v1/account/rotate-account-key`

Atomically swaps the recovery factor on an account. The client has generated a fresh account key, derived all dependent values, re-wrapped every gen of the DEK chain under {existing password KEK (unchanged — the password did not rotate), NEW recovery KEK}, and submits the bundle here.

Auth: JWT only — no step-up. The friction is client-side: the user must have generated and confirmed a new phrase before this endpoint is reachable. Adding a step-up gate here would just push it onto the toggle endpoints' auth machinery for no additional security gain — the rotation itself proves possession of the password (the new envelope decrypts only under the password KEK derived from the same password the JWT proves possession of).

Request body:

```
{
  "new_envelope":              "<full updated wrapped_data_key envelope (string)>",
  "new_recovery_lookup_key":  "<64-char hex>",
  "new_recovery_public_key":  "<base64 SPKI P-256>",
  "new_wrapped_account_key":  "<base64 ciphertext or null>"
}
```

`new_wrapped_account_key` reflects the current Model A/B state at rotation time:

- Model B → include the wrap (computed under DEK_gen1 + the v1 AAD against the new account-key plaintext). Rotation does NOT flip the model.
- Model A → set to `null` (or omit). The wrap stays NULL.

The SDK helper `tarn.accountKey.rotate()` reads `isStored()` to make this choice automatically.

Atomicity. The four fields (`wrapped_data_key` , `recovery_lookup_key` , `recovery_public_key` , `wrapped_account_key`) update in a single D1 statement. There is no partial-state window. After D1 commits, the credential blob is republished to Arweave (best-effort, in `waitUntil`).

Recovery-salt rotation. The protocol does NOT mandate that the salt change across rotation, but the SDK's `tarn.accountKey.rotate()` generates a fresh salt as part of the new envelope. Reasoning: rotation is an explicit "evict the recovery factor" operation, and refreshing the salt completes the eviction (a security-scoped attacker who recorded the old salt + ciphertext gains nothing against the new state). Old data wrapped under the old salt + old KEK remains on Arweave forever (immutable history) but is unwrappable without the now-defunct old phrase. Apps that drive the wire protocol directly may keep the old salt if they wish; the API stores whatever the client sends.

Response: `200 OK` with `{ "rotated": true }` . Audit row: `op = 'rotate'` .

Errors:

- `400` for invalid envelope / lookup key / public key / wrap shape.

- `400` if `new_recovery_lookup_key === credential_lookup_key` (the two identifier spaces must stay disjoint, mirroring the register / changeCredentials invariant).
- `409` if `new_recovery_lookup_key` collides with another account's recovery lookup key (astronomically unlikely with 256-bit HMAC output, but propagated cleanly so the SDK can surface a retry).
- `401` for missing/invalid JWT, `403` for app-role JWTs.

Effect on `recoverAccount`. Post-rotation, the OLD account key no longer authenticates `recoverAccount` (the OLD `recovery_lookup_key` no longer maps to any account row, so `/auth/challenge` returns 404). The NEW key works as expected. Pre-rotation data remains decryptable under either the password OR the new recovery factor (the DEK chain itself is unchanged; only the wrappings rotated).

Optional rotation in `recoverAccount`: `{ rotatePhrase: true }`

`recoverAccount` accepts an optional `rotatePhrase: boolean` parameter (default `false`). When `true`, the SDK runs `rotateAccountKey` inline after the recovery completes successfully and surfaces the new account key in the result object as `accountKey`. This is the "I think someone may have my recovery phrase too" escalation path during a forgot-password flow — the user gets fresh credentials AND a fresh account key in one round trip, with no separate UI step.

When `false` (default), `recoverAccount` behaves exactly as documented in §7a: the phrase stays the same after recovery; the result has no `accountKey` field. Existing apps see no behavior change.

The implementation reuses the rotation primitive directly — no new endpoint. If the inline rotation fails after a successful recovery, `recoverAccount` throws a partial-success error pointing at `tarn.accountKey.rotate()` for retry; the user is logged in under the new credentials and the OLD account key still works.

Passkey factor (Phase 6)

Phase 6 adds a third independent encryption factor: a WebAuthn passkey using the PRF (pseudo-random function) extension. A registered passkey can both unwrap the DEK chain (via a PRF-derived AES-KW key) and authenticate a session (via the standard WebAuthn signature against a stored public key).

Passkeys are **opt-in per account, opt-in per device**. An account with no registered passkeys behaves exactly as it did pre-Phase-6 — nothing on the wire changes, no new fields appear, all existing flows are untouched. An account may register zero, one, or many passkeys; each one independently unwraps the chain.

PRF derivation

The WebAuthn PRF extension lets the relying party hand the authenticator a 32-byte salt and receive back a deterministic 32-byte secret bound to (passkey, salt). Tarn runs that secret through HKDF-Expand with a fixed info string to derive the AES-KW wrapping key:

```
prf_output  = navigator.credentials.{create,get}({ extensions: { prf: { eval: { first:
    prf_salt } } } })
```

```
                .clientExtensionResults.prf.results.first       (32 bytes)

passkey_KEK = HMAC-SHA256(prf_output, "tarn-passkey-prf-v1" || 0x01)   (single-block
    HKDF-Expand)
```

The `prf_salt` is fresh random 32 bytes generated by the server at registration time and persisted in the `passkey_credentials` row alongside the credential's public key. Authentication reads the same salt back out so the PRF derivation produces the same secret deterministically.

Reusing the same passkey across different relying parties cannot collide because the PRF input includes the relying-party-id (browser-enforced). Reusing the same passkey within Tarn for a different purpose cannot collide either: the HKDF info string `"tarn-passkey-prf-v1"` namespaces the derivation away from any future PRF-based primitive.

`passkey_credentials` table

```
CREATE TABLE passkey_credentials (
  id           INTEGER PRIMARY KEY AUTOINCREMENT,
  account_id   TEXT NOT NULL,            -- accounts.data_lookup_key
  credential_id TEXT NOT NULL UNIQUE,    -- WebAuthn credential ID, base64url
  public_key   TEXT NOT NULL,            -- COSE-encoded public key, base64url
  prf_salt     TEXT NOT NULL,            -- 32 random bytes, base64url
  sign_count   INTEGER NOT NULL DEFAULT 0,
  device_label TEXT,                     -- user-supplied, ≤256 chars
  created_at   INTEGER NOT NULL,
  last_used_at INTEGER
);
```

`sign_count` is the WebAuthn replay counter. Tarn records it but does NOT enforce strict monotonicity — synced cross-device passkeys (iCloud Keychain, Google Password Manager) legitimately keep it at 0. Treat it as a soft signal.

Endpoints

- `POST /api/v1/auth/passkey/register-options` — JWT-authed (logged-in user). Returns `{ options, prf_salt }`. Server stores the registration challenge in `webauthn_challenges` (60 s TTL, single-use).
- `POST /api/v1/auth/passkey/register` — JWT-authed. Body: `{ credential, prf_salt, new_envelope, device_label? }`. Server verifies the WebAuthn registration response, stores the credential row, atomically replaces the envelope, republishes to Arweave. Returns `{ credential_id, device_label, created_at }`.
- `POST /api/v1/auth/passkey/authentication-options` — public. Returns `{ options, allow_credentials: [{ credential_id, prf_salt }, ...], rp_id }`. Server stores the authentication challenge in `webauthn_challenges` (60 s TTL).
- `POST /api/v1/auth/passkey/authenticate` — public. Body: `{ credential, previous_sid?, device_label? }`. Server verifies the assertion against the stored public key, increments sign_count, mints a session JWT. Returns `{ jwt, expiresIn, data_lookup_key, wrapped_data_key, account_key_stored, credential_id, stale_credential }`. The JWT carries `via_passkey: true` and `passkey_cred_id:`

`<credential_id>` claims so the refresh-credential endpoint can verify the caller is repairing the same credential they just signed in with.

- `POST /api/v1/auth/passkey/refresh-credential` — JWT-authed (passkey-side JWT only — `via_passkey: true` plus a matching `passkey_cred_id`). Body: `{ credential_id, new_envelope }`. Repairs a stale credential (see "Stale credentials and re-tap" below). The credential_id in the body MUST match the JWT's `passkey_cred_id`. The new envelope MUST include a `passkey_prf` wrapping for this credential at the latest gen — the server rejects envelopes that would re-establish the stale state. No step-up token is required: the passkey assertion that produced the JWT is the moral equivalent of a fresh re-authentication.
- `GET /api/v1/account/passkeys` — JWT-authed. Returns `{ passkeys: [{ credential_id, device_label, created_at, last_used_at, stale }, ...] }`. Public keys and PRF salts are NOT exposed (those are auth-internal). The `stale` boolean (Phase 6.2) is computed server-side per credential by parsing the live envelope and checking whether a `passkey_prf` wrapping exists for this `credential_id` at the latest gen — see "Stale credentials and re-tap" for the meaning. Surfacing it here lets a Settings UI render a "Refresh recommended" indicator without waiting for the user to bounce off a stale credential at login time.
- `DELETE /api/v1/account/passkeys/:credential_id` — JWT + step-up token. Body: `{ new_envelope }`. Strips the credential row and the matching wrappings, republishes to Arweave. Step-up reuses the existing `account_key_fetch` scope (the security posture is identical to the account-key toggles).

Stale credentials and re-tap (Phase 6.1)

A passkey credential is considered **stale** when the credential row exists in `passkey_credentials` but the LATEST gen of the account's envelope contains no `passkey_prf` wrapping for that credential_id. The model is intentionally derived from envelope shape alone — no side table, no schema change — so a `wrapped_data_key` republish from any source automatically updates the stale flag the next time the credential authenticates.

Staleness arises when an envelope-mutating operation creates a new gen and the SDK does not have the credential's PRF output to wrap the new gen DEK with. The canonical case is `changeCredentials`: the user is re-typing their password, the SDK has no opportunity to read the PRF output from any registered passkey, so the new gen ships without passkey wrappings. Phase 6.1 closes the gap by:

1. **Re-tap during `changeCredentials`.** When the account has registered passkeys, the SDK `changeCredentials` flow refuses to proceed without a `passkeyTapHandler` callback (silently producing stale credentials would be a worse UX failure). The handler is invoked once per registered credential; the user taps each authenticator they want to keep usable. The SDK fetches `/auth/passkey/authentication-options` for each credential, drives `navigator.credentials.get()` with the PRF extension, derives the wrapping key, and adds a `passkey_prf` wrapping to the new gen. Credentials whose tap is skipped or fails (authenticator unavailable, user dismissed) ship a new gen without their wrapping — they

become stale.

2. **Server-side stale detection on auth.** `/auth/passkey/authenticate` parses the live envelope after verifying the assertion, checks whether the latest gen has a `passkey_prf` wrapping for this `credential_id`, and sets `stale_credential: true` in the response when it does not. Authentication still succeeds (JWT minted, older gens unwrap normally) — the flag is purely informational so the SDK can transparently surface the repair.

3. **Stale-credential refresh.** When the SDK sees `stale_credential: true`, it can repair the credential inline. The flow:

   • Unwrap pre-stale gens via the passkey PRF KEK as usual.

   • Invoke a caller-supplied `stalePasskeyHandler` callback that prompts the user for both their username AND their password and resolves with `{ username, password }`. The handler returns both fields because passkey-only sessions don't have either cached on the client (the user has only ever tapped — never typed). Asking for both makes the repair work unconditionally; resolving with `null` aborts the repair.

   • Re-derive the password KEK from `(username, password, app_id)`, unwrap the latest gen via the password factor, re-wrap it under the passkey PRF KEK.

   • Build a refreshed envelope and POST to `/api/v1/auth/passkey/refresh-credential` with `{ credential_id, new_envelope }`.

   When no handler is supplied, the SDK throws `StalePasskeyError` instead — the app catches it and prompts re-registration via `tarn.passkeys.register()` from a password-authenticated session as the fallback recovery path.

   The same `stale` state is also surfaced proactively in the `GET /account/passkeys` response (per-credential `stale: boolean` field), so apps can render a "Refresh recommended" indicator in the Settings list before the user ever hits a stale credential at login time.

OS-synced passkeys. For the dominant case — Apple iCloud Keychain and Google Password Manager — all of a user's devices share the same credential_id and PRF secret. So a single re-tap on one device emits a wrapping that any of the user's other devices can derive themselves on next use. Real-world cross-device PRF stability is an empirical property of the platform sync layer, not something Tarn can guarantee from the wire spec; this assumption is documented for v1, with a path to per-device fallback (re-register) if a sync mismatch ever surfaces in practice.

Multiple distinct credentials. When a user has multiple credentials that don't share PRF secrets (e.g., iPhone Face ID plus a YubiKey, registered as separate credentials), they can only re-wrap credentials whose authenticator is physically present at change time. Per-credential staleness is independent — re-tapping one doesn't affect the other. Each stale credential has its own per-credential repair flow on next use.

RP-ID and origin handling

The relying-party-id is derived from the request's `Origin` header against a server-side allowlist (`getbookish.app` , `dev.getbookish.app` , `tarn.dev` , plus `localhost` for dev). A request from an unrecognized origin is rejected at the route boundary. The allowlist mirrors the CORS handler — keep them in sync when adding a new front-end host.

Audit

All passkey lifecycle events (`passkey_register` , `passkey_authenticate` , `passkey_remove`) write rows into the existing `account_key_fetch_log` table (the post-Phase-4 unified account-security audit table) so a Settings UI can show "Passkey added on X, last used Y, removed Z".

Library choice

`@simplewebauthn/server` v13 handles the CBOR/COSE parsing and signature verification on the server. `@simplewebauthn/browser` v13 provides the small client wrapper that converts the JSON-friendly options blob into the BufferSource fields `navigator.credentials.create/get` expect. Both satisfy the supply-chain rule (latest >7 days old at time of work).

Data lookup key

Generated by the API at registration. Random, unique, opaque 64-char hex string. Not derived from any client secret. Returned to the client at registration.

Security Model

Threat model

Tarn assumes credentials are never compromised. Credential changes are a convenience feature (e.g., switching the username to a new identifier), not a security remediation tool. All encrypted data is publicly visible on Arweave — security depends entirely on password entropy + Argon2id cost.

Forward secrecy on credential change

On every credential change a fresh random DEK is appended to the chain at gen N+1; future writes go to gen N+1. An attacker who later compromises old credentials can:

1. Derive the old `credential_lookup_key`
2. Find the old credential mapping blob on Arweave
3. Unwrap the OLD `wrapped_data_key` with the old `credential_encryption_key`
4. Recover DEKs at gens 1..N (the chain entries that existed at the time of the old credential blob)

What they CANNOT do: decrypt content written under gen N+1 (or later). Those CEKs are wrapped under the new DEK, which is not derivable from old credentials. The old credential blob does not contain the new DEK — it predates it.

What's still inherent to immutable storage: the old credential blob itself stays on Arweave forever, so an attacker who compromised the old credentials at any point retains permanent access to data written before the rotation. Tarn cannot revoke past leaks.

Password requirements

Since Arweave data is publicly available (encrypted), the security boundary is password entropy + Argon2id cost. The API enforces minimum password complexity at registration. The Argon2id parameters (m=64 MiB, t=3, p=1) follow the OWASP recommendation for memory-hard password hashing.

Per-app isolation

Different apps derive completely independent key sets from the same username+password. A compromise of one app's credential_lookup_key reveals nothing about the user's identity in another app. Even Arweave observers cannot link a user's Bookish account to their Cellar account.

What the API knows vs. doesn't know

API sees	API never sees
public_key (for signature verification)	private_key (signing)
credential_lookup_key	master_key
data_lookup_key (it generated it)	username, password
wrapped_data_key (opaque, AES-KW)	data_encryption_key
encrypted data blobs (opaque)	credential_encryption_key
app_id (which app the user registered for)	plaintext of any data blob

No secrets stored by the API. Zero-knowledge preserved for all key material that protects user data.

App Identity

Apps are first-class Tarn entities. An app must be registered before users can create accounts for it.

App registration

Privileged operation (initially manual, eventually a developer portal):

```
App generates ECDSA P-256 key pair
Tarn operator registers: { app_id, public_key }
Stored in D1 and on Arweave: Type: 'app-reg', Lk: app_id
```

App authentication

Same challenge-response protocol as users:

```
POST /api/v1/auth/challenge  { credential_lookup_key: <app_id> }
POST /api/v1/auth/verify     { credential_lookup_key: <app_id>, nonce, signature }
-> JWT with { sub: <app_id>, role: 'app' }
```

App wallets for Arweave storage

Each app has its own Arweave/Turbo wallet (Ethereum-compatible) for signing DataItems. The wallet private key is stored as a Worker secret (`APP_SIGNING_KEY`). Turbo provides free uploads under 100KB; larger uploads consume the wallet's Turbo balance.

App scoping on user accounts

User accounts are per-app. Registration requires a valid `app` parameter referencing a registered app. The JWT includes `app: <app_id>` . Write endpoints validate that the `App` tag matches `jwt.app` . A user cannot write entries for an app they're not registered with.

Arweave Tag Scheme

All blobs on Arweave carry these tags:

Tag	Meaning	Example
App	App identifier	`bookish` , `winelist`
Type	Blob type	`cred` , `entry` , `acct`
Lk	Lookup key (64-char hex)	`a3f8...`
Enc	Encryption method	`aes-256-gcm` or `tarn-cek-1`
V	Protocol version	`0.4.0`

For credential blobs: `Lk` = `credential_lookup_key` . For data blobs: `Lk` = `data_lookup_key` .

Additional tags for versioning and key rotation (data blobs only):

Tag	Meaning
Prev	Txid of prior version (edits)
Op	Operation flag (`tombstone`)
Ref	Txid referenced by tombstone

Tag	Meaning
Gen	DEK generation that wrapped the per-content CEK

The `Gen` tag carries the integer generation number (e.g. `1`, `2`) that selects which DEK from the user's chain unwraps the blob's CEK.

Blob Format

All content blobs use the per-content CEK shape:

```
[magic: 5 bytes][wrapped_CEK: 40 bytes][IV: 12 bytes][ciphertext + GCM tag: N+16 bytes]
```

- `magic = 0x54 0x41 0x52 0x4e 0x02` — ASCII `"TARN"` followed by format-version byte `0x02`. Future revisions reuse the `0x54 0x41 0x52 0x4e` prefix and increment.
- `CEK` is generated fresh per blob (`crypto.getRandomValues(32)`). `wrapped_CEK = AES-KW(CEK, DEK[currentGen])` — 40 bytes (32-byte CEK + 8-byte AES-KW overhead).
- `IV` is 12 random bytes per encryption.
- Ciphertext is AES-256-GCM with the per-blob CEK over UTF-8-JSON plaintext.

Per-blob fixed overhead: 73 bytes (5 + 40 + 12 + 16).

The generation indicator does not live in the blob bytes — it travels alongside as the Arweave `Gen` tag (e.g. `Gen: 1`, `Gen: 2`). This keeps the byte layout stable so a recipient who only has the CEK (sharing protocol) can skip bytes 5..44 and decrypt bytes 45..end without parsing tags.

Reading. The owner reads bytes 5..44 (`wrapped_CEK`), unwraps it with `DEK[gen]` (gen from the `Gen` tag), and decrypts bytes 45..end with that CEK. A recipient with a CEK from a share log skips the wrapped portion and decrypts directly.

The `Enc` tag for content blobs is `tarn-cek-1`.

> ***Note on the prior legacy format.*** *Earlier accounts wrote* `[IV: 12 bytes][ciphertext + tag]` *blobs (direct AES-GCM under the DEK,* `Enc: aes-256-gcm`*, no magic prefix). That format was supported on read for backward compat and dropped during the same cleanup that collapsed the envelope versions. Clients now reject any blob without the magic prefix.*

Credential Mapping on Arweave

The credential mapping blob is **not encrypted**. It contains:

```
{
  "data_lookup_key": "<64-char hex>",
  "wrapped_data_key": "<v1 envelope JSON — see Wrapped Data Key wire format above>",
  "public_key": "<base64-encoded ECDSA P-256 SPKI public key>",
```

```
  "app": "<app_id>",
  "recovery_lookup_key": "<64-char hex>",
  "recovery_public_key": "<base64-encoded ECDSA P-256 SPKI public key>",
  "wrapped_account_key": "<base64 ciphertext — Model B only, omitted for Model A>"
}
```

Every account carries a recovery factor, so `recovery_lookup_key` and `recovery_public_key` are always present. `wrapped_account_key` (Phase 3) appears for Model B accounts and is omitted for Model A; it is opaque to the server and decryptable only by the user (gen-1 DEK + AAD `"tarn-wrapped-account-key-v1"`).

No value here is secret. Storing unencrypted enables:

- Single API call for registration (no second client round-trip)
- Full API self-healing from Arweave (all account fields recoverable)
- No asymmetric crypto complexity for negligible privacy gain

Write Authorization Rules

Write access is controlled by a **rules** array on each user account. At write time, the API evaluates all rules — **every rule must pass**. Unknown rule types **fail closed** (deny).

Rules are set by authenticated app identities. The entire rules array is **replaced** on each update (no merging).

Rule types (v1)

`max_entries` — user has fewer than N entries matching the filters

```
{ "type": "max_entries", "limit": 1000, "since": "2026-04-01T00:00:00Z", "app":
    "bookish", "entry_type": "entry" }
```

`max_bytes` — this individual entry is smaller than N bytes

```
{ "type": "max_bytes", "limit": 102400 }
```

`expires` — the current time is before this timestamp

```
{ "type": "expires", "at": "2027-04-01T00:00:00Z" }
```

Rule evaluation

```
For each rule: unknown type -> DENY; evaluates false -> DENY
All rules pass -> ALLOW
No rules (NULL) -> DENY (accounts require rules to be set by the app)
```

Note: with per-app accounts, the default for new accounts is DENY (no rules set). The app must set rules after the user registers. For free-tier apps, set `[{ "type": "max_entries", "limit": 5 }]` immediately after user creation.

API State

D1 `accounts` table:

```
credential_lookup_key TEXT PRIMARY KEY,
public_key TEXT NOT NULL,
data_lookup_key TEXT NOT NULL UNIQUE,
wrapped_data_key TEXT NOT NULL,
app TEXT NOT NULL,
rules_json TEXT,
created_at INTEGER NOT NULL,
recovery_lookup_key TEXT,    -- nullable; UNIQUE when present (issue #12)
recovery_public_key TEXT     -- nullable (issue #12)
```

D1 `apps` table:

```
app_id TEXT PRIMARY KEY,
public_key TEXT NOT NULL,
created_at INTEGER NOT NULL
```

D1 `entries` table (authoritative store for live state; see "D1 authority" below):

```
txid TEXT PRIMARY KEY,
app TEXT, type TEXT,
lookup_key TEXT,
prev_txid TEXT, is_tombstone INTEGER, tombstone_ref TEXT,
block_timestamp INTEGER, tags_json TEXT, cached_at INTEGER
```

D1 authority

Tarn is the sole write path for any data associated with a (`data_lookup_key` , `app` , `type`) tuple. Every successful write synchronously upserts into D1 before returning success to the client. As a consequence:

- **D1 is authoritative for live state.** Once a tuple has been bootstrapped, reads are served entirely from D1. Tarn does not periodically re-scan Arweave — there is no other writer to reconcile against.
- **Bootstrap markers** (stored in `cache_meta`) record that a tuple has been ingested. Set by the write path on first write, or by the read path on first read if no marker exists.
- **Arweave GraphQL is consulted only on cold bootstrap:** a first read for a tuple Tarn has never processed. After bootstrap, the marker short-circuits all subsequent queries.
- `block_timestamp` **/ confirmation status** is set at write time (NULL = pending) and no longer updates once the marker is set. Clients that need Arweave confirmation status can check Turbo directly for a given txid.

Rebuild from Arweave: All tables fully rebuildable from Arweave blob scans. After rebuild, `cache_meta` markers are cleared so the next read for each tuple re-bootstraps from the restored `entries` rows.

Flows

1. Register (new account)

```
CLIENT (local):
  1. Derive master_key from username + password (Argon2id,
    salt=SHA-256(normalizedUsername))
  2. Derive credential_lookup_key, credential_encryption_key, signing_key_pair
     (all include app_id in HKDF info)
  3. Generate recovery: phrase = BIP39 24 words, recovery_salt = 16 random bytes,
     phrase_entropy → recovery_lookup_key + recovery_signing_key,
     recovery_KEK = Argon2id(phrase, recovery_salt)
  4. data_encryption_key (DEK) = crypto.getRandomValues(32)
  5. wrapped_data_key = v1 envelope:
       { v: 1, kdf: 'argon2id', kdf_params, recovery: { kdf_params, salt },
         dek_chain: [{ gen: 1, wrappings: [
           { factor: 'password',        wrapped: AES-KW(DEK, credential_encryption_key)
    },
           { factor: 'recovery_phrase', wrapped: AES-KW(DEK, recovery_KEK) },
         ]}] }

CLIENT -> API:
  6. POST /api/v1/auth/register
     Body: { credential_lookup_key, public_key, wrapped_data_key, app,
             recovery_lookup_key, recovery_public_key, share_pub, share_lookup_key }

API:
  6. Verify app is registered in apps table (-> 400 if not)
  7. Check credential_lookup_key not already in use (-> 409 if exists)
  8. Generate data_lookup_key (random, unique)
  9. Store in D1: { credential_lookup_key, public_key, data_lookup_key,
    wrapped_data_key, app }
  10. Persist credential mapping to Arweave
  11. Return: { data_lookup_key }
```

2. Sign in (existing account, any device)

```
CLIENT (local):
  1. Derive master_key, credential_lookup_key, credential_encryption_key,
    signing_key_pair
     (all include app_id in HKDF info)

CLIENT -> API:
  2. POST /api/v1/auth/challenge { credential_lookup_key }
     Returns: { nonce, data_lookup_key, wrapped_data_key }

CLIENT (local):
  3. Sign nonce with private_key -> signature
  4. data_encryption_key = AES-KW-Unwrap(wrapped_data_key, credential_encryption_key)

CLIENT -> API:
  5. POST /api/v1/auth/verify { credential_lookup_key, nonce, signature }
     Returns: { jwt }
     JWT contains: { sub: data_lookup_key, role: 'user', app: app_id }
```

3. Create data

```
CLIENT -> API:
  POST /api/v1/entries [JWT]
  Tags: { App, Type, Lk: data_lookup_key, Enc, V }
  Body: encrypted_blob

API:
  1. Verify JWT
  2. Check App tag matches jwt.app
  3. Check Lk tag matches jwt.sub (data_lookup_key)
  4. Evaluate write authorization rules -> 403 if denied
  5. Build + sign DataItem, cache in D1, upload to Turbo in background
  6. Return: { txid, status: 'pending' }
```

4. Retrieve data

```
GET /api/v1/entries?app={app}&type={type}&key={data_lookup_key}
No auth required. IP rate-limited.
Returns resolved entries (tombstones applied, Prev-chains resolved).
Client downloads + decrypts blobs from Arweave gateways.
```

5. Update data

```
PUT /api/v1/entries/{prior_txid} [JWT]
Tags include Prev: prior_txid. API verifies ownership. Old entry superseded.
```

6. Delete data

```
DELETE /api/v1/entries/{target_txid} [JWT]
Tags include Op: tombstone, Ref: target_txid. Entry hidden by resolution.
```

7. Change credentials

```
CLIENT (authenticated with old credentials, holds DEK chain DEK[1..N]):
  1. Derive NEW keys from new username + password (same app_id)
  2. Mint DEK[N+1] = crypto.getRandomValues(32)
  3. Re-wrap every gen under the new password KEK; preserve old gens' recovery
     wrappings byte-for-byte (AES-KW is deterministic, so re-wrapping under
     the same recovery_KEK would be a no-op anyway). When the caller supplies
     `phrase`, derive recovery_KEK and add a recovery wrapping to gen N+1 too.
  4. Phase 6.1 — passkey re-tap. When the account has registered passkeys,
     prompt the user to re-tap each authenticator (caller-supplied
     `passkeyTapHandler`). For each successful tap, derive the PRF wrapping
     key and add a passkey_prf wrapping for that credential_id to gen N+1.
     Credentials skipped at change-time or whose authenticator is absent
     ship without a new-gen wrapping → marked "stale" by the server on next
     authenticate. Apps with registered passkeys MUST supply the handler
     (the SDK refuses to silently produce stale credentials).
  5. new_wrapped_data_key = v1 envelope with chain entries:
       [ { gen: i, wrappings: [
             { factor: 'password',        wrapped: AES-KW(DEK[i],
    new_credential_encryption_key) },
             // recovery wrapping per existing gen, plus optional gen N+1 if phrase
    supplied
             { factor: 'recovery_phrase', wrapped: <preserved or fresh> },
             // passkey wrappings: preserved verbatim per existing gen,
             // plus per-credential wrappings on gen N+1 for re-tapped credentials
             { factor: 'passkey_prf', credential_id: ..., wrapped: ... }, ...
         ]}
         for i in 1..N+1 ]
  6. Future writes use Gen=N+1.

CLIENT -> API:
  PUT /api/v1/auth [JWT from old credentials]
  Body: { new_credential_lookup_key, new_public_key, new_wrapped_data_key,
          new_share_pub, new_share_lookup_key }

data_lookup_key unchanged. Existing data untouched. Old gens stay readable;
new writes go to the new gen. Skipped passkeys become stale on the new gen
and surface a refresh path on next authenticate (see Passkey factor §
"Stale credentials and re-tap").
```

7a. Account recovery (via account key)

When the user has lost their password (or wants a security-grade reset, per the design-doc positioning of recovery as the response to suspected compromise):

```
CLIENT (local — only the account key + new credentials):
  1. phrase_entropy = BIP39.mnemonicToEntropy(phrase)
  2. recovery_lookup_key  = HMAC(phrase_entropy, "tarn" || "recovery-lookup" || app_id
     || "1" || 0x01)
  3. recovery_signing_key = ECDSA P-256 from HMAC(phrase_entropy, "tarn" ||
     "recovery-sign" || app_id || "1" || 0x01)

CLIENT -> API:
  4. POST /api/v1/auth/challenge { recovery_lookup_key }
```

```
     Returns: { nonce, data_lookup_key, wrapped_data_key }   (the existing v1 envelope)

CLIENT (local):
  5. Parse v1 envelope → recovery_salt + KDF params
  6. recovery_KEK = Argon2id(phrase, recovery_salt, params)
  7. Unwrap DEK chain via FACTOR_RECOVERY_PHRASE
  8. Sign nonce with recovery_signing_key.privateKey

CLIENT -> API:
  9. POST /api/v1/auth/verify { recovery_lookup_key, nonce, signature }
     Returns: { jwt }   (JWT carries via_recovery: true)

CLIENT (local):
  10. Derive new password keys from (newUsername, newPassword)
  11. Re-wrap entire DEK chain under {new_password_KEK, recovery_KEK} factors
      (preserving the existing recovery salt; recovery wrappings are byte-identical
       to the originals by AES-KW determinism)
  12. Build v1 envelope from re-wrapped chain

CLIENT -> API:
  13. PUT /api/v1/auth { new_credential_lookup_key, new_public_key,
                         new_wrapped_data_key,
                         new_recovery_lookup_key,
                         new_recovery_public_key }
     (recovery_lookup_key + recovery_public_key are unchanged by recovery — phrase is
    the same;
      we re-send them so a corrupted server-side row would self-heal.)

data_lookup_key unchanged. All pre-recovery data is decryptable under the new
    credentials.
```

Optional `{ rotatePhrase: true }`**.** Pass this on the SDK `recoverAccount` call to bundle a phrase rotation into the same flow — useful when the user is recovering specifically because they suspect the phrase itself has been compromised. After step 13 above, the SDK runs `rotateAccountKey` inline and surfaces the new phrase in the result as `accountKey`. The OLD phrase no longer authenticates `recoverAccount` post-call. Default `false` preserves the existing semantics: recovery rotates credentials, the phrase stays the same. See the `rotateAccountKey` section for details.

8. D1 Recovery

All tables fully rebuildable from Arweave. Entries self-heal on cache miss. Accounts rebuilt from `Type=cred` blobs. Apps rebuilt from `Type=app-reg` blobs.

9. Delete account

```
DELETE /api/v1/auth [JWT]
Tombstones credential mapping on Arweave. Deletes D1 account row.
Data remains encrypted on Arweave permanently.
```

10. Set user write rules (app -> API)

```
PUT /api/v1/accounts/{data_lookup_key}/rules [JWT with role: 'app']
Body: { rules: [...] }
Replaces rules_json. Persists to Arweave as Type: 'app-config'.
```

Client <-> API Summary

Auth endpoints

```
POST /api/v1/auth/register
  Body: { credential_lookup_key, public_key, wrapped_data_key, app }
  Auth: none
  Returns: { data_lookup_key }
  Errors: 400 (unregistered app), 409 (credential_lookup_key in use)

POST /api/v1/auth/challenge
  Body: { credential_lookup_key }
     OR { recovery_lookup_key }   (recovery flow)
  Auth: none
  Returns: { nonce, data_lookup_key, wrapped_data_key }
  Errors: 404 (unknown lookup key)

POST /api/v1/auth/verify
  Body: { credential_lookup_key, nonce, signature }
     OR { recovery_lookup_key, nonce, signature }   (recovery flow)
  Auth: none (signature IS the auth — verified against the public key matching the
    lookup key type)
  Returns: { jwt }   (JWT carries via_recovery: true when the recovery_lookup_key path
    is used)
  Errors: 401 (invalid/expired nonce, bad signature)

PUT /api/v1/auth
  Body: { new_credential_lookup_key, new_public_key, new_wrapped_data_key,
          new_recovery_lookup_key?, new_recovery_public_key? }
  Auth: JWT (works with both regular login JWTs and via_recovery: true JWTs)
  Returns: 200 OK
  Errors: 401, 409 (new credential_lookup_key OR new_recovery_lookup_key already in use)

DELETE /api/v1/auth
  Auth: JWT
  Returns: 200 OK
  Errors: 401

POST /api/v1/auth/step-up
  Body: { credential_lookup_key, nonce, signature, scope: "account_key_fetch" }
  Auth: signature (verified against the row's stored public_key, same as /auth/verify)
  Returns: { step_up_token, expires_at, scope }
  Errors: 400 (unknown scope), 401 (bad signature / expired nonce)

GET /api/v1/account/account-key
  Auth: JWT + X-Step-Up-Token (single-use, account_key_fetch scope)
  Returns: { wrapped_account_key, recovery_salt, kdf_params, recovery_lookup_key }
  Errors: 401 (missing/invalid auth), 404 with body {error:"no_account_key_stored"}
    (Model A)

PUT /api/v1/account/account-key
  Body: { wrapped_account_key }
  Auth: JWT + X-Step-Up-Token (account_key_fetch scope)
  Returns: { stored: true }
```

```
  Errors: 400 (invalid wrap), 401 (missing/invalid auth), 403 (app role)

DELETE /api/v1/account/account-key
  Auth: JWT + X-Step-Up-Token (account_key_fetch scope)
  Returns: { stored: false }  OR  { stored: false, already_disabled: true } (idempotent
    on Model A)
  Errors: 401 (missing/invalid auth), 403 (app role)

POST /api/v1/account/rotate-account-key
  Body: { new_envelope, new_recovery_lookup_key, new_recovery_public_key,
    new_wrapped_account_key }
  Auth: JWT
  Returns: { rotated: true }
  Errors: 400 (invalid bundle), 409 (recovery_lookup_key collision), 401, 403
```

Recovery-kit delivery and rendering are not Tarn responsibilities

Tarn intentionally does **not** expose a recovery-kit transport endpoint. Earlier protocol drafts included `POST /api/v1/recovery/email` as a "no-storage, brief in-memory visibility" forwarder; that endpoint has been removed. Even ephemeral handling of plaintext account-key material on Tarn-operated infrastructure was a violation of the zero-knowledge framing the rest of the protocol enforces, and any operational compromise (logs, supply-chain, subpoena, future bug introducing persistence) would have exposed the most sensitive payload in the entire system.

Tarn also does not render recovery kits. The SDK exposes the account-key string (and the gen-1 DEK, when Model B is in use); apps render their own kit format and decide how to surface it to the user (download, print, app-operated transport). The historical in-SDK PDF renderer has been removed; apps that want a PDF render one with their own toolchain. This keeps the SDK bundle smaller and gives apps full control over branding and layout.

App endpoints

```
PUT /api/v1/accounts/{data_lookup_key}/rules
  Body: { rules: [...] }
  Auth: JWT with role: 'app'
  Returns: 200 OK
  Errors: 401, 403

GET /api/v1/status
  Auth: JWT (user or app)
  Returns: { users, entries, apps, wallet, protocol_version, timestamp }
```

Data endpoints

```
POST /api/v1/entries
  Headers: Authorization: Bearer <jwt>, X-Arweave-Tags: [...]
  Body: <encrypted blob bytes>
  Auth: JWT
  Returns: { txid, status: 'pending' }
  Errors: 401, 403 (Lk/App mismatch or rules deny), 413

POST /api/v1/entries/batch
  Body: { entries: [{ data: "<base64 encrypted bytes>", tags: [...] }, ...] }
  Auth: JWT
  Returns: { entries: [{ txid, gateway }], count, status: 'pending' }
  Errors: 401, 403, 413
  Max 100 entries per batch. Counts as 1 rate-limit hit.
  Rules evaluated once for the entire batch (max_entries checks count + batchSize).

GET /api/v1/entries?app={app}&type={type}&key={data_lookup_key}
  Auth: none (IP rate-limited)
  Returns: { entries: [{ txid, tags }], total }

PUT /api/v1/entries/{prior_txid}
  Auth: JWT
  Returns: { txid, prevTxid, status: 'pending' }

DELETE /api/v1/entries/{target_txid}
  Auth: JWT
  Returns: { txid, tombstoneRef, status: 'pending' }
```

Design Decisions

Why HKDF-Expand (HMAC) for sub-key derivation

Raw `SHA-256(key || domain)` is vulnerable to length-extension attacks. HMAC (which is HKDF-Expand for a single output block, per RFC 5869) is immune by construction. The structured info string `protocol || purpose || app_id || version || counter` replaces ad-hoc domain strings with a defined, versioned format that naturally accommodates per-app isolation.

Why AES-KW for key wrapping

AES-KW (RFC 3394) is purpose-built for key wrapping: deterministic (no IV management), minimal overhead (8 bytes), built-in integrity checking, and available in WebCrypto natively via `wrapKey` / `unwrapKey`. AES-GCM works but is not the specialist tool.

Why per-app account isolation

Same username+password produces independent identities per app via the `app_id` in the HKDF info string. This prevents cross-app data leakage, cross-app session sharing, and cross-app identity correlation on Arweave. It also means apps must be registered before users can create accounts, which closes the "anyone can freeload on Tarn's Arweave wallet" gap.

Why app validation at registration

Write endpoints check `App` tag against `jwt.app`. The JWT's `app` claim is set at login based on the account's registered app. Unregistered apps cannot create accounts, therefore cannot get JWTs, therefore cannot write data. This is enforced at the identity layer, not the write layer.

Why default rules are DENY

New accounts have `rules_json = NULL`, which means DENY. The app must explicitly set rules after user creation (e.g., free tier: `max_entries=5`). This prevents orphaned accounts from writing unlimited data on the app's Arweave wallet.

Why reads are unauthenticated

Data is on Arweave permanently (encrypted). Auth on reads only protects the cache layer. A permanent, auditable, API-independent data export page must be possible.

Why a fresh random DEK at registration

Earlier versions of Tarn self-wrapped the credential_encryption_key as the DEK at registration, redundantly but uniformly. Issue #11 replaced this with a fresh `crypto.getRandomValues(32)` DEK at gen 1. This is a prerequisite for forward-secret DEK rotation (different gens must be independent random keys, not derived from credentials) and for the planned recovery-phrase factor (different KDFs must be able to wrap the *same* DEK independently).

Why per-content CEKs

Each blob is encrypted with its own freshly-generated 32-byte CEK; the CEK is wrapped under the current generation's DEK and prepended to the blob. This adds 73 bytes per blob but enables granular access control for the future sharing protocol (a CEK can be shared to another user without exposing the DEK), per-content rotation on revocation, and simpler capability delegation. The 5-byte magic prefix `0x54 0x41 0x52 0x4e 0x02` makes the format unambiguously distinguishable from legacy direct-DEK blobs.

Why the generation tag lives in Arweave tags, not in the blob

Section 5 (sharing protocol) requires that a recipient holding only a CEK can decrypt the content portion of a blob without knowing or parsing the owner-side wrapped CEK header. Encoding the generation in the blob bytes would shift the content offset and break that invariant. Putting the generation in an Arweave `Gen` tag keeps bytes 5..44 reserved for the wrapped CEK and bytes 45..end as CEK-only-decryptable content — independent of who's reading.

Sharing primitives — terminology

The full sharing protocol (HPKE handshake, per-pair stealth-addressed share log, identity rotation, revocation) is specified in `notes/2026-04-28-tarn-sharing-design.md`. Section 6 of the implementation plan (issue #18) renamed the public surface from "friend" to "connection" so the SDK is product-neutral. The on-the-wire protocol identifiers are now:

Layer	Identifier
HPKE info (request)	`tarn-connection-request-v1`
HPKE info (accept)	`tarn-connection-accept-v1`
Inbox tag prefix	`tarn-connection-inbox-v1-`
Connections record content_id	`tarn-connections-v1`
Pending requests record content_id	`tarn-pending-requests-v1`
Server-recognized inbox blob types	`connection-request-v1`, `connection-accept-v1`

The persisted connections record's inner array field is `connections: [...]` (was `friends: [...]` in the design doc).

Muted-connections record (issue #18)

The mutual-connection primitive is symmetric — both sides see each other's share-log entries by default. Apps that want a Strava-style asymmetric "follow" feel build it on top of the mutual primitive plus a per-side mute filter. The SDK persists the filter as an encrypted Tarn data blob:

```
content_id = "tarn-muted-connections-v1"
plaintext  = JSON({
  app_id:  <string>,
  version: 1,
  muted:   [{ share_pub: B(32 bytes), muted_at: <unix_seconds> }, ...]
})
```

DEK-encrypted via the per-content CEK pattern (same as the connections + pending-requests records). Stored on Arweave as a normal `tarn-share-state` entry with `Eid=tarn-muted-connections-v1`, so it syncs across the user's own devices via the existing data-blob storage path. No new D1 schema, no API change.

Semantics — explicitly local:

- **Per-side, per-user.** Mute is set by the muting party only. The muted party receives no signal and continues publishing share-log entries to the muting party's outbound log as normal. Only the muting party's *view* changes.
- **No protocol-level effect.** Mute does NOT alter what the muted party can see, what they can publish, or what tags either side polls. It is purely a UI hint.
- **App-driven filtering.** `readShareLog` and `syncShareLog` do NOT short-circuit on muted connections. Apps still need programmatic access to muted-connection state (e.g., to render a "Muted" tab). The SDK exposes `isMuted(connection)` and `listMutedConnections()` so apps can filter at the call sites that should be filtered (the main feed) without losing access at the call sites that shouldn't (the muted tab).

SDK surface: `tarn.muteConnection(c)` / `tarn.unmuteConnection(c)` / `tarn.listMutedConnections()` / `tarn.isMuted(c)`.

Session persistence (Section 7, issue #19)

A logged-in `TarnClient` holds a bag of derived secrets in memory: the password-derived signing keypair, the unwrapped DEK chain, the X25519 sharing keypair, and the JWT. When the page closes, this state is lost — the user must re-enter their password (paying the full Argon2id cost) on every tab open and every browser restart.

For consumer apps this is a UX floor that's hard to ship below. Section 7 adds an explicit, opt-in session-persistence primitive that lets apps trade a bounded increase in attack surface for the ability to restore the in-memory state without re-deriving from password.

Why it must be opt-in

Persisting derived keys to client-side storage is a meaningful change to Tarn's threat surface. Today, a same-origin XSS on a Tarn app can act as the user **for as long as the page is open** — once the tab closes, the keys are gone. With persistence enabled, the same XSS gains the ability to either decrypt the persisted blob in-page (calling the SDK on the live origin) or, if combined with a localStorage/IndexedDB exfil to a remote server, replay the session **only while the wrapping key remains valid** (see at-rest encryption below).

This is a real escalation, even with a non-extractable wrapping key. Apps with stricter postures should not opt in. The default (`new TarnClient(api, appId)` followed by `login()`) does not persist anything; apps must explicitly call `serializeSession()` to enable persistence.

Threat model

What persistence costs:

1. **Same-origin XSS becomes pseudo-persistent.** An attacker who lands code execution on the origin can call `resumeSession()` on the persisted blob and act as the user up to the blob's `expiresAt`. This is bounded by the 7-day max age and by the absence of refresh-on-use (see lifecycle).
2. **No server-side revocation in v1.** A user who suspects their device is compromised cannot tell the API "log out all sessions." The mitigation is `changeCredentials()` — which rotates the signing key and the DEK chain, and the SDK clears the IndexedDB wrapping key as a side effect, rendering all previously-emitted blobs on this origin unreadable. v2 will add per-session identifiers and an explicit revoke endpoint so a user can invalidate sessions without changing credentials.
3. **Cross-tab attack visibility.** All same-origin tabs can read each other's IndexedDB and localStorage. An attacker on one tab can resume the session on another. This is identical to the existing same-origin XSS surface and is not new.

What persistence does NOT cost:

1. **Cross-origin attackers** see no change. The wrapping key is bound to origin via IndexedDB scoping; the persisted blob is unreadable off-origin.
2. **Network attackers** see no change. The persisted blob never leaves the device.
3. **Tarn API operators** see no change. The persisted blob is never sent to the API.

Wire format (pre-encryption JSON)

The plaintext payload below is what gets encrypted under the at-rest wrapping key. Apps MUST treat the resulting ciphertext as opaque — schema fields are subject to change in future versions and are documented here only so the threat model can be reasoned about.

```
{
  "v": 1,
  "createdAt": <unix-seconds>,
  "expiresAt": <unix-seconds>,
  "apiBase": "https://api.tarn.dev",
  "appId": "bookish",
  "username": "user@example.com",
  "dataLookupKey": "<64-char hex>",
  "credentialLookupKey": "<64-char hex>",
  "kdfVersion": 2,
  "envelopeVersion": 4,
  "currentGen": 2,
  "dekByGen": [
    { "gen": 1, "rawBytes": "<base64 32 bytes>" },
    { "gen": 2, "rawBytes": "<base64 32 bytes>" }
  ],
  "signingPrivateKey": "<base64 PKCS#8 DER>",
  "signingPublicKey": "<base64 SPKI>",
  "sharingPrivateKey": "<base64 32 bytes>",
  "sharingPublicKey": "<base64 32 bytes>",
  "recoveryFactorMeta": null | {
    "salt": "<base64 16 bytes>",
    "kdfParams": { "m_kib": ..., "t": ..., "p": ... },
    "wrappingsByGen": [{ "gen": 1, "wrapped": "<base64>" }, ...]
  },
  "jwt": "<jwt or null>"
}
```

The `credential_encryption_key` is intentionally NOT persisted — once the DEK chain is unwrapped at login, the KEK becomes dead state on the client (no code path reads it post-login). Re-deriving it would require the password, which we do not have. Persistence carries the unwrapped DEKs directly, sidestepping the KEK entirely. The recovery factor's `wrappingsByGen` is preserved verbatim (per-gen AES-KW ciphertext bytes) so a subsequent `changeCredentials()` can re-emit the recovery wrappings without requiring the user to re-enter the phrase, matching the in-memory `#recoveryFactorMeta` semantics.

At-rest encryption

The plaintext schema above is encrypted under an AES-256-GCM wrapping key managed by the SDK and stored in IndexedDB (database `tarn-session`, object store `keys`, record id `wrapping-key-v1`). The wrapping key is created with `extractable: false` so its raw bytes never enter JS userland — even if XSS reads the IndexedDB store, it can only invoke the key for decrypt (which still grants in-page session takeover) but cannot exfiltrate it for offline replay on another device or origin.

```
on-disk blob (returned from serializeSession, base64url-encoded):
  IV (12 bytes) || AES-256-GCM ciphertext+tag

wrapping key (in IndexedDB, non-extractable):
  AES-256-GCM, generated on first persist, scoped to origin
```

The SDK creates the wrapping key on first call to `serializeSession()`. Subsequent calls reuse it. `clearSession()` deletes the IndexedDB record, which renders all previously-emitted blobs on this origin unreadable.

Lifecycle

1. **Creation.** App authenticates via `login()`, `register()`, or `recoverAccount()`. App calls `await client.serializeSession()` and stores the returned blob (typically in `localStorage`).
2. **Resume.** On a subsequent page load, app calls `await TarnClient.resumeSession(apiBase, appId, blob)`. If the blob is well-formed, decryptable on this origin, schema-version `1`, and not past `expiresAt`, returns a logged-in `TarnClient`. Otherwise returns `null` — caller falls back to the login UI.
3. **Expiry.** Hard 7-day max age, baked into the blob's `expiresAt` field at creation. The SDK does NOT auto-refresh expiry on use — a "fresh" blob is only emitted by an explicit `serializeSession()` call. This bounds the worst-case window of a quiet exfil-and-replay attack.
4. **Invalidation events.** `changeCredentials()`, `recoverAccount()`, and `deleteAccount()` all rotate the signing key and the DEK chain. The SDK clears the IndexedDB wrapping key on each, rendering all previously-emitted blobs on this origin unreadable. Apps that want continued persistence must re-call `serializeSession()` after these events.
5. **Explicit logout.** `await client.clearSession()` deletes the IndexedDB wrapping key. Subsequent `resumeSession()` calls return `null` for any pre-existing blob.

The `expiresAt` field is enforced client-side. A motivated attacker with code execution on the origin could in principle tamper with the field before it's encrypted (the SDK is in their JS context; they can call any of its functions). v1 accepts this — the threat is upper-bounded by the existing in-page-XSS exposure, which is itself the dominant risk this section is documenting.

SDK API surface

```
client.serializeSession(): Promise<string>
  Requires: client is authenticated.
  Returns:  opaque base64url ciphertext.
  Side effects: creates the IndexedDB wrapping key if absent; idempotent.
  Throws:   if the client is not authenticated.

TarnClient.resumeSession(apiBase, appId, blob): Promise<TarnClient | null>
  Returns:  a logged-in TarnClient, or null if the blob is expired,
            tampered, schema-mismatched, or unreadable on this origin.
  Never throws on bad blobs — null is the recoverable signal so apps can
  uniformly fall back to the login UI without distinguishing failure modes.

client.clearSession(): Promise<void>
  Deletes the IndexedDB wrapping key. Renders all previously-emitted
  blobs unreadable on this origin. Does not affect the in-memory
  client state — the caller can keep using the live client until it's
  garbage-collected.
```

Known v1 gaps

- **No server-side revocation.** Addressed in Section 7.5 below — per-session identifiers + revoke endpoints. Until 7.5 ships, "log out all devices" requires `changeCredentials()`.
- **No idle expiry.** Blobs expire only at the hard 7-day mark; an actively-used session has no separate idle timeout. Apps that want shorter idle windows can implement them by calling `clearSession()` from their own activity tracker.
- **Browser-only.** The IndexedDB + non-extractable WebCrypto storage strategy is browser-specific. Tarn-on-Node and Tarn-on-React-Native session persistence are deferred — the API surface (`serializeSession` / `resumeSession`) is intentionally storage-agnostic in shape so a future implementation can swap backends without changing the surface.
- **No share-log cache participation.** Persisted sessions hydrate share-log state on first read/sync from Arweave, just like fresh logins. Pure perf optimization, not a correctness gap.

Server-side session management (Section 7.5, issue #20)

Section 7 covers persistence on a single device — keeping a logged-in client alive across page reloads. Section 7.5 covers the complementary capability across devices: letting users see active sessions and revoke individual ones without performing a full credential change.

The combination closes the consumer-app session story. Apps render a "Manage devices" page; users kill an unwanted session with a click; the API enforces the revocation immediately.

Why server-side state at all

Today, JWTs are stateless: signed at `/auth/verify`, verified by signature + expiry on every authenticated request, with no D1 round-trip. This is the right default for performance and operational simplicity. The cost: there is no way to invalidate a JWT before its 15-minute TTL elapses.

Section 7.5 makes user-role JWTs stateful by attaching a per-session identifier (`sid`) and a corresponding row in a new D1 `sessions` table. Revoking a session = deleting the row. Subsequent authenticated requests carrying that JWT fail at the middleware layer.

App-role JWTs (`role: 'app'`) stay stateless. Apps are not session-tracked in v1 — per-app revocation is a separate concern handled at the app-registry level.

D1 schema

New migration `0013_sessions.sql` :

```
CREATE TABLE IF NOT EXISTS sessions (
  sid TEXT PRIMARY KEY,
  data_lookup_key TEXT NOT NULL,
  app TEXT NOT NULL,
  created_at INTEGER NOT NULL,
  last_seen_at INTEGER NOT NULL,
  device_label TEXT,
  via_recovery INTEGER NOT NULL DEFAULT 0
);
CREATE INDEX IF NOT EXISTS idx_sessions_dlk ON sessions(data_lookup_key);
CREATE INDEX IF NOT EXISTS idx_sessions_last_seen ON sessions(last_seen_at);
```

`sid` is a UUID (v4). `device_label` is nullable and app-supplied at `/auth/verify` time (see below). `via_recovery` mirrors the JWT's `via_recovery` claim so the user can see in their session list whether a session was created via the recovery flow.

JWT changes

User-role JWTs gain a `sid` claim:

```
{ "sub": "<data_lookup_key>", "role": "user", "app": "<app_id>", "sid": "<uuid>",
    "iat": ..., "exp": ... }
```

App-role JWTs unchanged. `via_recovery: true` JWTs (issue #12) also carry `sid` .

`/auth/verify` body changes

```
POST /api/v1/auth/verify
Body: {
  credential_lookup_key | recovery_lookup_key,
  nonce,
  signature,
  previous_sid?: string,        // optional — see "session continuity" below
  device_label?: string,        // optional — max 64 chars; stored on new sessions only
}
```

Session continuity across JWT refreshes

The SDK's `#requireAuth` silently re-authenticates when the JWT expires. Without continuity, every refresh would create a new session row — ~96 rows/day per active device under typical use.

To prevent that, `/auth/verify` accepts an optional `previous_sid` :

- **Present and active for the same `data_lookup_key`:** server reuses the sid, updates `last_seen_at`, returns a fresh JWT with the same `sid`. No new row.
- **Absent, expired, or revoked:** server mints a new sid, inserts a new row. Returns the new sid in the JWT.

The SDK extracts `sid` from its current JWT (decoding the payload, no signature check needed since the SDK trusts its own state) and passes it as `previous_sid` on every re-verify. Result: one row per device-installation per app, persisting until either the user revokes it or the lazy-prune step (next paragraph) removes it as stale.

Lazy pruning

At the start of `/auth/verify`, before minting a new sid, the server runs:

```
DELETE FROM sessions
WHERE data_lookup_key = ?1 AND last_seen_at < ?2
```

with `?2 = now - 24h`. This bounds the table to "sessions used in the last 24h" per account. A user who hasn't logged in for a day gets a clean slate; an active user keeps their existing rows.

New endpoints

```
GET /api/v1/sessions
  Auth: JWT (user role)
  Returns: [
    {
      sid: <uuid>,
      created_at: <unix_seconds>,
      last_seen_at: <unix_seconds>,
      device_label: <string | null>,
      via_recovery: <boolean>,
      is_current: <boolean>,
    },
    ...
  ]
  Sorted by last_seen_at descending. is_current is true for the row matching
  the calling JWT's sid. Empty array is a valid response (e.g., session
  pruned between calls).

DELETE /api/v1/sessions/:sid
  Auth: JWT (user role)
  Returns: 204 No Content on success.
  Errors: 401 (no JWT or stale sid), 404 (sid not found OR belongs to a
          different account — no existence leak across accounts).

DELETE /api/v1/sessions
  Auth: JWT (user role)
  Query: ?except=current — optional; preserve the calling sid.
  Returns: 204 No Content. Without ?except=current, the calling JWT's sid
           is also deleted; the next request from this client will 401.
```

All three endpoints scope by `data_lookup_key` extracted from the JWT. A user cannot list, see, or revoke sessions belonging to another account.

Auth middleware: stateful verification

User-role JWTs with a `sid` claim now require an additional D1 check:

```
1. Verify JWT signature + exp (existing)
2. Extract sid from claims; if absent (pre-7.5 JWT), grandfather — see migration
3. Look up sid in sessions table (with isolate-level cache, see below)
4. If row absent → 401
5. Update last_seen_at via ctx.waitUntil() (off the hot path, throttled — skip
   if existing last_seen_at is < 60s old)
```

Isolate-level cache

The naïve implementation runs a D1 SELECT on every authenticated request. To amortize: each Worker isolate keeps a small `Map<sid, { active: boolean, cachedAt: number }>` with a 5-second TTL.

- **Hit (within TTL):** answer from memory; no D1 read.
- **Miss or stale:** D1 SELECT, update cache.
- **Revoke endpoints:** the isolate handling the revoke updates its own cache entry to `active: false` immediately; other isolates serve stale until the 5-second TTL expires.

Net revocation latency: ≤5 seconds globally, immediate on the originating isolate. The 5-second window is acceptable — the threat model already accommodates same-origin-XSS-equivalent risk for the lifetime of the JWT (15 min). A 5-second cross-isolate inconsistency is well within that envelope.

The cache is in-memory only; no KV, no Durable Object. It evaporates on isolate eviction (no persistence concern) and is naturally bounded by the working set of recent sids.

Pre-7.5 migration

Rolling deploy:

1. Apply the D1 migration creating the `sessions` table.
2. Deploy the Worker. From this moment, new JWTs from `/auth/verify` carry `sid` and create rows. Existing in-flight JWTs (no `sid` claim) are grandfathered — the auth middleware skips the sessions check when the claim is absent.
3. After ~15 minutes (the JWT TTL), all live JWTs carry `sid` and the grandfather path is dead code. It is left in place for safety (handles a clock skew or a long-lived test JWT); it's a no-op for production traffic.

No client-side migration is required. SDKs older than 7.5 continue to work — they receive sid-bearing JWTs from the new server but don't pass `previous_sid` on re-verify, so each refresh creates a new row. That's the "table bloat" path; the lazy-prune step keeps it bounded. SDK upgrade brings the client onto the continuity path.

Interaction with credential rotation

`changeCredentials`, `recoverAccount`, and `deleteAccount` all rotate the signing key. The new public_key invalidates old JWTs at signature-check time, so revoking sessions is technically redundant — but doing it explicitly is the cleaner mental model and removes any timing-window concern.

The API performs:

```
DELETE FROM sessions WHERE data_lookup_key = ?1
```

as part of each operation, before publishing the new credential blob. The SDK additionally calls `clearSession()` (Section 7) on the local device to wipe the persisted blob.

SDK API surface

```
client.listSessions(): Promise<Array<SessionInfo>>
  SessionInfo = {
    sid, createdAt, lastSeenAt, deviceLabel, viaRecovery, isCurrent
  }
  Auth: requires authenticated client. Returns sessions for the calling user.

client.revokeSession(sid: string): Promise<void>
  Auth: requires authenticated client. 404 surfaces as null (caller can no-op
  or refresh listSessions to reflect the now-removed row).

client.revokeAllSessions(): Promise<void>
  Revokes all sessions including the current. After this resolves, the SDK
  also calls clearSession() locally; the caller should treat the client as
  dead and prompt for re-login.

client.revokeOtherSessions(): Promise<void>
  Revokes all sessions except the current. Implemented by the SDK as a
  DELETE /api/v1/sessions?except=current. Current session continues working.
```

Plus optional `deviceLabel` parameters on the existing entry points:

```
client.register(username, password, { ..., deviceLabel?: string })
client.login(username, password, { deviceLabel?: string })
client.recoverAccount({ ..., deviceLabel?: string })
```

`deviceLabel` flows to `/auth/verify` and is stored on the freshly-created session row. It is not modifiable post-creation in v1; a future endpoint may add `PATCH /api/v1/sessions/:sid` for relabeling.

Known v1 gaps

- **24h pruning window.** The lazy-prune step deletes sessions older than 24h, so a user who logs in once and never returns has no surfacing in `listSessions` even if their JWT is still valid (it isn't — JWT TTL is 15 min). For long-term audit logging, a separate "session-history Arweave-pinned audit log" would be a different piece of work; not v1.
- **Revocation latency up to 5 seconds.** Bounded by the isolate cache TTL. Acceptable trade-off for amortized D1 reads. If a use case demands sub-second revocation, the cache

TTL can be lowered or disabled per-environment via a config.

• **No device-grouping heuristics.** The SDK's `previous_sid` continuity keeps it to one row per device-installation, but a single user with two browser profiles on the same physical machine has two rows. Apps that want to group by physical device must rely on app-supplied `device_label` to disambiguate.

• **App-role JWTs are stateless.** Per-app session tracking would require a separate `app_sessions` table and is out of scope for 7.5.

Invite tokens (Section 8, issue #22)

The connection-handshake primitives shipped under Sections 5a–5d let two users form an end-to-end-encrypted connection if and only if the sender already knows the recipient's username and the recipient is a Tarn user with `share_discoverable=true`. For a large class of consumer-app flows — "scan this QR to add me", "send this link to your friend in Slack", "register and click my invite link" — those preconditions don't hold. Section 8 adds a complementary primitive: **opaque, single-use, time-limited invite tokens** that let the inviter publish a redemption slot without knowing the recipient's identity, and let the recipient redeem it (potentially after signing up) without ever transmitting the inviter's identifier through the channel that carried the link.

Why server-mediated

Three alternative shapes were considered and ruled out:

1. **Stateless QR / link** — keys baked into the URL, no server state. Loses single-use; a leaked link in any messenger channel exposes the inviter to arbitrary stranger redemption forever.

2. **Pure-Arweave invite blob** — inviter publishes an Arweave entry; recipient reads it, sends a normal connection request. Single-use cannot be enforced at storage (Arweave is immutable); app-side single-use ("accept the first matching request, drop the rest") races between the legitimate recipient and any malicious party that scrapes the link.

3. **Reuse the existing inbox-tag mechanism** — the inbox is share_pub-keyed and time-windowed; an "anonymous inbox" without a share_pub doesn't fit the model, and we'd be inventing single-use semantics on a primitive that doesn't want them.

All three lose the atomic single-use property that only the server can provide cheaply. Section 8 is server-mediated for that reason and that reason alone — every other piece of the flow is client-side cryptography over an opaque blob.

Cryptographic shape

The inviter generates two independent 32-byte secrets client-side:

• `token_id` — opaque server-side index. 256-bit URL-safe random. Sent to the API in the path; the server stores ciphertext keyed on it.

• `payload_key` — AES-256-GCM key that encrypts the invite payload. **Never transmitted to the API.** Carried only in the URL fragment, which browsers do not include in HTTP requests.

```
plaintext = JSON({
  inviter_share_pub:    <base64url 32 bytes>,
  inviter_signing_pub:  <base64url SPKI>,
  app_id:               <string>,
  issued_at:            <unix seconds>
})
ciphertext = IV(12) || AES-256-GCM(plaintext, payload_key, IV) + tag
on-wire    = base64(ciphertext)        # what `payload` field carries
url        = <apps.invite_url_template, with {token_id}> # base64url(payload_key) in
    the URL fragment
                                                          # see "App-side URL template"
    below
```

The payload carries only the inviter's public-key material plus scope/timestamp metadata. There is no display-name slot on the wire — Tarn has no concept of a user-facing name in its identity model, so the protocol does not pretend to provide one. Apps that want to render "X invited you" UI pass that name through their own delivery channel (e.g., the message accompanying the invite link).

The server stores `(token_id, payload, ...metadata)` and never learns the contents of `payload` — `inviter_share_pub` and `inviter_signing_pub` are not visible to the API. This is the consistent zero-knowledge story across the rest of the protocol.

App-side URL template

There is no Tarn-hosted landing page. Apps own the URL surface. A new column on the `apps` table records each app's invite URL template:

```
ALTER TABLE apps ADD COLUMN invite_url_template TEXT;
-- e.g. "https://app.bookish.example/invite/{token_id}"
-- {token_id} is the only supported substitution.
```

Set at app onboarding. The SDK calls `GET /api/v1/apps/:app_id/invite-template` (or returns it from `createInviteToken` directly) and constructs the final URL by substituting `{token_id}` and appending `#<base64url(payload_key)>` as the fragment.

The app's web handler at that URL is responsible for:

- Reading `token_id` from the path and `payload_key` from `window.location.hash.slice(1)`.
- Calling `tarn.previewInviteToken(token_id, payloadKey)` to confirm the invite is still valid (returns scope + fingerprint + timestamps; no inviter name — apps render "Maya invited you" UI from whatever the inviter put in the message accompanying the link).
- Calling `tarn.redeemInviteToken(token_id, payloadKey)` once the recipient is authenticated (signing up first if needed).
- Server-side rendering the page with `<meta property="og:title">` etc. for messenger-preview unfurls — the app can fetch the unauthenticated preview server-side to populate them.
- Universal Links / App Links handling for native apps.

Tarn's surface stays the protocol primitive plus a per-app URL template. The UX is the app's.

D1 schema

```
CREATE TABLE invites (
  token_id TEXT PRIMARY KEY,            -- 256-bit URL-safe random, client-generated
  app_id TEXT NOT NULL,
  inviter_dlk TEXT NOT NULL,            -- inviter's data_lookup_key
  payload BLOB NOT NULL,                -- opaque AES-GCM ciphertext
  issued_at INTEGER NOT NULL,           -- unix seconds
  expires_at INTEGER NOT NULL,          -- unix seconds; default issued_at + 7d, max 30d
  used_at INTEGER,                      -- NULL until redeemed
  redeemer_share_pub_fingerprint TEXT   -- short hex; populated on redeem for sender
    display
);

CREATE INDEX idx_invites_expires ON invites(expires_at);
CREATE INDEX idx_invites_inviter ON invites(inviter_dlk);
```

`payload` is byte-stable for the inviter's session (the same `(plaintext, payload_key)` always wraps to the same ciphertext given a fixed IV; the IV is fresh per invite, so different invites have different bytes — this property doesn't matter to retry idempotency since `token_id` is the primary key).

Lazy cleanup (no cron)

Tarn has no cron infrastructure (`auth_nonces` and `write_rate_limits` both use lazy expiry; we follow the precedent):

- `previewInvite` and `redeemInvite` filter by `expires_at` in the SELECT — expired rows behave as 410 Gone regardless of physical presence.
- `redeemInvite` opportunistically issues `DELETE FROM invites WHERE expires_at < ?1` on ~5% of calls (the same probabilistic-cleanup pattern used by `write_rate_limits`).

The 24-hour grace before deletion documented in the original RFC is unnecessary under this model — expired-but-undeleted rows are filtered out by the SELECT side.

Endpoints

`POST /api/v1/invite` (authenticated, user-role)

```
Body: { token_id, app_id, payload, expires_at }
  - token_id: 43-char base64url
  - payload:  base64 of IV || ciphertext+tag (max 4 KiB on the wire)
  - expires_at: unix seconds; server enforces ≤ now + 30 days
  - app_id must equal the JWT's app claim

Returns 201: { token_id, expires_at }
Errors:
  400 if expires_at > 30d in the future, or token_id malformed
  409 if token_id already exists (caller retries with fresh random)
  413 if payload > 4 KiB
  429 if inviter has exceeded the rate limit (see below)
```

`GET /api/v1/invite/:token_id` **(unauthenticated)**

```
Returns 200: { app_id, payload, issued_at, expires_at }
  payload is the opaque ciphertext (base64). Recipient decrypts client-side.
Errors:
  404 if not found
  410 if expired
  409 if already used (used_at IS NOT NULL)
  429 if per-IP preview rate exceeded
```

The app_id and timestamps are unencrypted — they're already implied by the URL hosting the invite, and the recipient's app needs `app_id` to confirm scope before redeeming. Everything else (inviter_share_pub, signing_pub) is inside the encrypted payload.

`POST /api/v1/invite/redeem/:token_id` **(authenticated, user-role)**

```
Body: { redeemer_share_pub_fingerprint }
  - redeemer_share_pub_fingerprint: short hex (e.g., 8 chars) of the
    recipient's share_pub, recorded for the inviter's listIssuedInvites
    display.

Atomicity: UPDATE invites SET used_at = ?1, redeemer_share_pub_fingerprint = ?2
           WHERE token_id = ?3 AND used_at IS NULL AND expires_at > ?4
         If affected_rows = 0:
           - Lookup the row to determine error mode
           - 410 if expired, 409 if already used, 404 otherwise

Returns 200: { app_id, payload, issued_at, expires_at }
  Same shape as preview; recipient decrypts and proceeds with the
  connection-request HPKE handshake.
Errors: 404 / 410 / 409 / 429 as above
```

The redemption response includes the same payload as preview because the server can't tell the difference — both responses serve opaque ciphertext. The server-side state change is `used_at` being set atomically.

Rate limiting

Existing primitives, no new infrastructure:

- **Create** — `checkWriteRateLimit`-style D1 atomic counter, key `invite-create:<dlk>:<hour>`, max **10/hour per inviter**. Atomic via `INSERT ... ON CONFLICT DO UPDATE`.
- **Redeem** — same shape, key `invite-redeem:<dlk>:<hour>`, max **50/hour per redeemer**. Bounded to detect runaway redemption clients.
- **Preview** — `checkAndIncrementRateLimit` (KV), key `invite-preview:<ip-hash>:<hour>`, max **100/hour per IP**. Defense-in-depth against token-id enumeration; the 256-bit token space already makes brute-force infeasible.

All three fail open on rate-store outage (consistent with existing patterns).

Connection.label primitive

The current connections record stores `share_pub`, `signing_pub`, `username`, `established_at`, `initial_request_nonce`. The `listConnections` JSDoc (client/src/tarn.js) lists `label?: string` as an optional field, but no code reads or writes it — vestigial.

Connection record back-compat

Older connection records on Arweave were written with an `email` field instead of `username`. The SDK reads both: when deserializing a connection record, an `email` field is mapped to `username` if no `username` field is present. New writes use `username` exclusively. Apps and recovery clients should treat the two as the same field, with `username` as the canonical name and `email` as a legacy alias accepted on read only.

Section 8 makes it a first-class primitive:

- `acceptConnectionRequest(nonce, { label })` — optional label set at accept time.
- `setConnectionLabel(connection, label)` — relabel an existing connection.
- `listConnections()` returns `label` per entry (always present in the type, may be `null`).
- The label persists in the connections record (`tarn-connections-v1`) blob alongside other per-connection fields. No new server-side surface.
- The label is local to the labeling user — the labeled party does not see what they were labeled.

Used by invite redemption to seed the new connection's label with the `label` the inviter recorded against the matching token in `tarn-issued-invites-v1` — the inviter answered "who am I sending this to?" at invite-creation time, and the auto-accept path applies that answer to the connection that forms. The recipient never sees the label; it's the inviter's local annotation.

Auto-accept on the inviter side

After the recipient redeems and sends a normal connection request back, the inviter's app sees an inbound request with a new optional field, `via_invite_token: <token_id>`. The SDK's `listIncomingRequests` surfaces this; the SDK's auto-accept logic on the inviter side:

1. Receives the inbound request.
2. **Reads** `tarn-issued-invites-v1` **fresh from the API** (no in-memory cache — the issuer may have created the invite on a different device, and that device's write hits D1 synchronously, but the receiving device's in-memory state can be stale).
3. If `via_invite_token` matches an entry in `tarn-issued-invites-v1`, auto-accepts (no UI prompt).
4. If no match, the request stays in the pending list as a normal incoming request — the user sees it and decides. (Not silently dropped; that would lose legitimate redemptions during edge-case scenarios.)

The `tarn-issued-invites-v1` blob is the standard encrypted-data-blob pattern (mirrors `tarn-muted-connections-v1`, `tarn-connections-v1`, etc.):

```
content_id = "tarn-issued-invites-v1"
plaintext  = JSON({
  app_id:  <string>,
  version: 1,
  invites: [
    {
      token_id, label, issued_at, expires_at,
      redeemed_at, redeemer_share_pub_fingerprint
    },
    ...
  ]
})
```

DEK-encrypted. Synced across the inviter's devices via standard data-blob storage. Pruned locally once the matching connection is established or the entry expires.

Threat model

• **Leaked invite link** (screenshot, accidental Slack post). Mitigations: 7-day default expiry, hard 30-day max, single-use, sender-visible `redeemer_share_pub_fingerprint` so inviter can detect surprise redemption and revoke + reissue.

• **Malicious recipient redeems but refuses the handshake**. Token consumed; inviter must reissue. No worse than the username-handshake case where a recipient declines.

• **Server compromise**. Attacker can enumerate live tokens via the `invites` table but cannot decrypt payloads (the `payload_key` lives only in URL fragments held by recipients). Best they can do is mark tokens used (denial-of-service) or redirect connection-request handshakes (which fail signature verification on the inviter side because the attacker doesn't have the recipient's identity to sign as).

• **Token enumeration**. 256-bit space is brute-force-infeasible regardless. The 100/hour/IP preview rate-limit is defense-in-depth.

• **Inviter-identity exposure to API operators**. Operators see `(inviter_dlk, token_id, time, redeemer_fingerprint)` per row. They do **not** see `inviter_share_pub` or `inviter_signing_pub` — those are inside the encrypted payload.

SDK surface

```
client.createInviteToken({
  label?: string,               // local-only label for the connection-to-be, ≤ 64 chars
                                //   stored in tarn-issued-invites-v1; never sent to
    the recipient
  expiry_days?: number,         // default 7, server max 30
}): Promise<{
  token_id: string,
  invite_url: string,           // app's template + #fragment, ready to share
  expires_at: number,
}>

client.previewInviteToken(token_id, payloadKey): Promise<{
  inviter_share_pub_fingerprint: string,   // short hex for verification UI
  app_id: string,
  issued_at: number,
  expires_at: number,
} | null>                       // null on expired / used / not found

client.redeemInviteToken(token_id, payloadKey): Promise<Connection>
                                // Throws on expired / used / not found / app_id
    mismatch.
                                // The returned connection becomes live once the
    inviter's
                                // session auto-accepts the inbound request.

client.listIssuedInvites(): Promise<Array<IssuedInvite>>

client.revokeIssuedInvite(token_id): Promise<void>
                                // App-side delete from tarn-issued-invites-v1, plus a
                                // best-effort DELETE /api/v1/invite/:token_id on the
    server.

client.acceptConnectionRequest(nonce, { label? }): Promise<Connection>   // updated
client.setConnectionLabel(connection, label): Promise<void>              // new
```

`listConnections` return type gains `label: string | null`.

Known v1 gaps

- **No multi-redeem variant.** All v1 invites are single-use. A `max_redemptions: N` flag for "invite link to a book club" use cases is plausible follow-up but has a different threat model and is deferred.
- **No revocation propagation.** `revokeIssuedInvite` clears the inviter's local record and best-effort deletes from the server. A malicious recipient who already retrieved (but not yet redeemed) the payload via preview can still attempt redemption; the server-side delete prevents it from succeeding. Acceptable.
- **No anonymous-inbox-style metadata privacy.** Server learns `(inviter_dlk, time)` for every issued invite and `(token_id, redemption_time, IP)` for every redemption. Acceptable disclosure given the tight binding to the inviter's account; documented in Publicly observable metadata below.

• **App-onboarding.** Setting `invite_url_template` is part of the existing manual app-registration flow; no developer-portal self-service in v1.

Publicly observable metadata

Tarn protects content (and the keys protecting content) end-to-end, but a number of metadata properties are observable to anyone who knows enough to ask. Apps building on Tarn should be aware of what's visible vs. what's protected, and surface this honestly to users where relevant.

What's protected (encrypted; not observable to anyone without keys):

• All content blobs (book entries, etc.) — encrypted under per-content CEKs, only owner + explicitly-shared recipients can decrypt.

• All share-log entries (per-pair shared content) — encrypted under per-pair `K_AB`.

• All connection-handshake payloads (request + accept) — HPKE-sealed to recipient's `share_pub`.

• All friend-graph relationships — neither Tarn nor an Arweave observer can extract who is connected to whom from the protocol alone.

• All account keys — generated client-side. In Model A (no backup stored) Tarn never holds any form of the key; in Model B Tarn holds only the DEK-encrypted ciphertext, which is opaque without password-derived keys.

What's publicly observable (no keys needed):

• **Connection-request inbox metadata.** `GET /api/v1/share/inbox/fetch` is **unauthenticated by design** — a recipient who lost their JWT (e.g., still booting on a fresh device) needs to be able to poll their inbox. Anyone who knows a user's `share_pub` can:

  • Compute their inbox tag for any time window (the tag is `HMAC(H(share_pub), "tarn-connection-inbox-v1-" || app_id || "-" || window)` — derivable from public info)

  • Fetch the sealed-but-non-decryptable HPKE blobs at that tag

  • Observe the **volume + timing** of incoming connection requests for that user

  The contents stay encrypted; only HPKE recipient (the user) can open them. But "user X received N connection requests on day D" is publicly extractable.

• **Account existence via discoverability.** If a user has `share_discoverable=true`, anyone who knows their username can look up their `share_pub` via `GET /api/v1/share/lookup`. This confirms the username is a registered Tarn user. If `share_discoverable=false`, the lookup returns `share_pub: null` — but existing connections already cached `share_pub` from the original handshake.

• **Per-recipient activity from share-log writes.** Tarn's per-tag uniqueness check at `POST /api/v1/share/log/publish` means an observer querying tag-existence can confirm specific tags are taken. Tags are pseudorandom (HMAC under per-pair secret), so this doesn't reveal

relationships, but bulk-enumeration of common patterns isn't ruled out at scale.

- **Aggregate volume + timing on Arweave.** All Tarn-bundled writes are signed by the Tarn-bundler wallet on Arweave. An Arweave observer can see "Tarn bundler activity per hour" but cannot link writes to individual users without protocol-level knowledge.

Acceptable residual leaks documented for v1 (per sharing design §11.5):

1. **Username-based discoverability lookup leaks "user A is interested in user B"** at handshake time. Once connected, all subsequent traffic is unlinkable — the per-pair tags are stealth-addressed.
2. **Tarn-side correlation** of write/read timing per session may allow Tarn (the operator) to infer some relationships statistically. Mitigation deferred (would require dummy traffic, mixing networks, etc.).
3. **Per-recipient inbox metadata** (the bullet above) — accepted because the unauthenticated fetch is a hard requirement for fresh-device recovery flows.

App-side guidance:

Apps building on Tarn should communicate in user-facing privacy docs that:

- Connection-request **timing and volume** can be observed by anyone who has seen the user's `share_pub` (typically: anyone they've shared their username with, plus accepted connections).
- Setting `share_discoverable=false` prevents new strangers from discovering the user's `share_pub` via username lookup, but does not retroactively hide it from anyone who already cached it.
- **Content** is protected by end-to-end encryption; only the people the user explicitly shares with can read what they share.

For most use cases (private reading lists, friend-circle apps), these properties are appropriate trade-offs. For higher-stakes sensitive data, additional mitigations (decoy traffic, alternative discovery flows) would be needed; sharing roadmap §14.2 + §14.7 covers that direction.

Cryptographic References

- **HKDF:** RFC 5869 — HMAC-based Extract-and-Expand Key Derivation Function
- **AES-KW:** RFC 3394 — Advanced Encryption Standard Key Wrap Algorithm
- **PBKDF2:** RFC 8018 — Password-Based Key Derivation Function 2
- **AES-GCM:** NIST SP 800-38D — Galois/Counter Mode
- **ECDSA P-256:** FIPS 186-4 — Digital Signature Standard
- **Pattern:** Bitwarden-style architecture (PBKDF2 → master key → sub-keys → wrapped data key) with LUKS-style key slot wrapping

Open Questions

1. **Protocol versioning:** The `V` tag and the `version` field in the HKDF info string allow future derivation changes (e.g., PBKDF2 → Argon2id). Migration path needs design.

2. **Rate limiting on registration:** Registration is unauthenticated. IP rate limiting sufficient for now. CAPTCHA or proof-of-work at scale.

3. **Multiple credential mappings after changes:** Old credential mappings persist on Arweave. On rebuild, latest wins. Minor information leak (credential change history via shared data_lookup_key).

4. **App wallet monitoring:** Turbo balance monitoring and top-up. Operational concern, not protocol.